

预测误差与波动电价下的微网购电和储能调度（Version 2）

摘要

光伏和负载预测误差会同时影响计划购电、储能状态与紧急购电，预测精度的提高未必带来电费下降。本文建立统一的能量与费用账本，并对确定性、日前不确定性、日内更新和波动电价四种条件逐层建模。

针对第一问，在允许无成本弃电的条件下，证明充放电循环可以在保持购电量与全部储电状态不变的情况下消除，因而线性规划可给出与互斥整数模型同等费用的物理解。单日费用 35,126.95 元，较无储能降低 26.90%。针对第二问，保留因果岭回归与 70%历史误差余量，334 天费用 14,132,612.16 元；降低名义日末储能目标的候选在一月较优、全年反而较差，未替换原策略。

针对第三问，以发布版本匹配的历史净负载误差补充 50%分位数候选，一月选择 12:00 及 18:00 更新、日末名义下限 6000 kWh、不作额外日内负载校正。全年费用降至 13,765,317.70 元，相比原策略降低 2.29%，紧急购电量为 166,980.76 kWh。针对第四问，保留日前岭回归策略，采用相同改进的滚动模型与一月选择的同期价格预测，费用为 14,486,341.42 元，较原滚动策略降低 2.06%。

两项滚动改进与各自原策略的期末储电量相同，费用改善并非来自净消耗电池库存。保留 GRU 三种种子与集成的预测对照，按发布时刻分解误差，同时公开未改善的候选与事后表现更好的消融。研究属于同一年度数据上的模型修订与顺序回测，不构成新的独立年度验证。

关键词：微网调度；线性规划；误差校准；滚动优化；储能

一 问题重述

题面研究小区光伏、负载、储能与外部电网之间的电力调控 [1]。储能可以在低价或光伏富余时吸收电量，在高价或供电不足时释放电量；调度既要满足负载，也要尽可能节省购电费用。本文依次研究确定性调度、预测误差、日内更新与波动电价四个层次。

第一问给定每天重复的电价、负载和单日光伏预测，要求每天 00:00 制定全天购电计划，且日初日末储电量相同。需报告指定十分钟时段的购电量、全天总量和费用，以及四小时充放电汇总。第二问中负载与光伏逐日变化，正常购电仍按计划量收费，供电缺口通过五倍交易电价的紧急购电补足，需输出 2—12 月每日完整策略及四个指定日期的结果。

第三问进一步利用每日四个发布时刻的光伏预报，决定是否以及何时调整未执行的购电计划，并计入增购和减购费用。第四问使用附件中的实际波动电价，比较不同价格预测器在日前与滚动两类策略下的实际费用。

二 问题分析

第一问是已知输入曲线下的跨时段资源分配。线性回归不负责决定何时充电；购电、电池和负载之间的能量约束才是主模型。加入二元变量后可直接保证同一时段不同时充放电，避免事后修复改变费用与可行性。

第二问的预测误差具有不对称经济后果：低估净负载可能触发五倍电价补购，高估则可能产生仍需付费的富余计划电量。因此先预测负载与光伏，再构造风险余量并求购电计划，最后用实际观测执行与计费。预测误差用于评价预测器，实际购电总费用用于评价完整策略，两者不能互相替代。

三 模型假设与数据处理

3.1 参数及工作假设

统一参数见表 1。功率上限按微网母线侧定义，所有决策电量使用 kWh。题面“充放电效率为 90%”可能存在单程与往返解释差异，主实验采用两侧各 90%，并对另一解释单独进行敏感性分析。

表 1 储能参数与时间离散

参数	数值	性质
额定容量	12000 kWh	题面给定
允许储电量	1200—10800 kWh	题面给定
最大充放电功率	5000 kW	题面给定
初始储电量	6000 kWh	1 月 1 日 00:00 给定
充电 / 放电效率	0.9 / 0.9	主实验解释
时段长度	1/6 h	十分钟离散

(1) 将每个功率值解释为以时间标签为右端点的十分钟区间平均值。由此原始 00:10 对应 00:00—00:10，原始 00:00+1 对应 23:50—24:00。结果副本按该时间轴统一标签；该解释并非题面明确说明的采样定义。

(2) 不设置题面未给出的售电收入与折旧价格，允许不能使用的电量被弃用。第一、二问普通购电按计划量结算，富余计划量不退款；第三、四问主动减购另按第八节的调整账本结算。电池磨损成本没有纳入当前目标。

(3) 实际执行近似每十分钟内功率恒定、当前供需可即时观测并平衡。该近似只服务于区间内的充放电和紧急购电，不允许日前读取当天真实曲线。

(4) 第二问实际 SOC 连续跨日。日前名义轨迹的日末下限取 6000 kWh，作为防止计划耗空电池的策略设置；不强制实际日末回到 6000，也不按天重置电池。

3.2 输入审计与时间边界

附件 1 有 144 个时段，附件 2 的负载与光伏各有 $365 \times 144 = 52,560$ 个观测。读取后检查到完整的 2025 年日期、相同的负载和光伏时间轴，未发现缺失、负数或重复日期，未无依据删除峰值。附件 1 混合存储了时间类型与字符串，统一转换分钟后核对 10、20、…、1440。电量按下式换算：

$$L_{d,t} = P_{d,t}^{\text{load}} \Delta t, \quad V_{d,t} = P_{d,t}^{\text{PV}} \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

重新读取原始工作簿确认：附件 1 的时间列含 60 个时间类型单元格和 84 个字符串，均已规范化；附件 2 的两类序列各 52,560 项无缺失、非有限值或负数，且无重复日期或完全重复的日曲线。全局四分位距规则未标记超界点，但这一检查不能证明不存在局部测量误差。

因此，本次预处理没有执行填补、删点、缩尾或平滑，只执行时间格式与单位规范化。光伏有 23,540 个零值，其本身不是缺失标记，保留在原始记录与回测中。预测值的非负截断属于模型输出规则，训练集标准化属于特征处理，均不等同于修改实际观测。

实际数据的测量不确定性没有单独给出，因此不添加虚构置信区间。全文图线来自原始输入或保存的实际计算结果，时段内不做平滑插值。

四 符号说明

下文沿用表 2 的符号。单日模型省略日期下标，日内区间用 1—144，状态包含 145 个边界点；跨日计算另带日期下标。

表 2 主要符号及单位

符号	含义	单位
d, t	日期与十分钟时段	—
$L_{d,t}, V_{d,t}$	负载与光伏电量	kWh
$\hat{L}_{d,t}, \hat{V}_{d,t}$	日前负载与光伏预测	kWh
$g_{d,t}, e_{d,t}$	计划购电与紧急购电	kWh
$c_{d,t}, d_{d,t}$	母线侧充电与放电	kWh
$w_{d,t}$	不能利用的富余电量	kWh
$S_{d,t}$	区间起点的储电量	kWh
p_t	固定分时交易电价	元/kWh
η_c, η_d	充电与放电效率	—
z_t	充电状态二元变量	—
λ, q	岭惩罚强度与余量分位数	—
$r_{d,t}(q)$	非负净负载预测余量	kWh

日期下标 d 与放电变量的字母 d 通过下标位置区分；为简化实现，程序字段分别使用 `date` 和 `discharge_kwh`。

五 第一问的模型建立与求解

5.1 供需结构与决策目标

已给曲线及固定电价如图 1 所示。光伏集中在白天，而负载覆盖全天，储能调度应同时响应供需差额和分时价格。电价使用独立面板，避免功率与价格共用坐标造成误读。

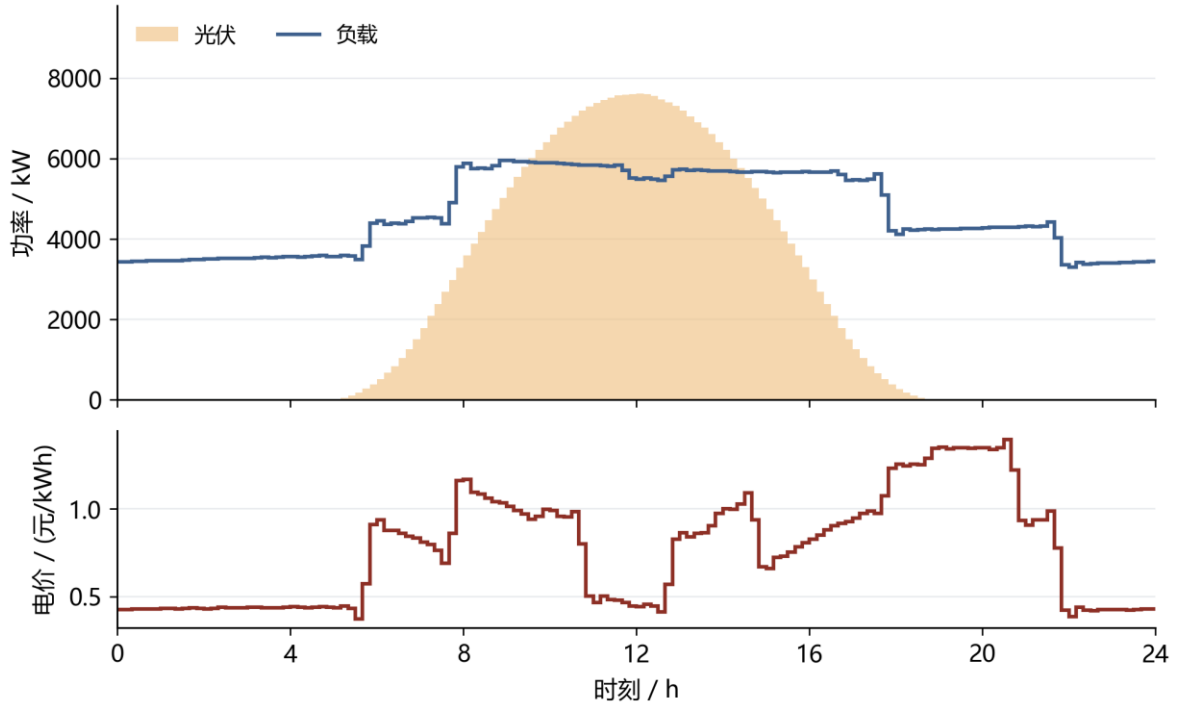


图 1 第一问供需曲线与分时电价

图中每一阶梯对应一个十分钟平均功率或电价，光伏阴影为功率曲线下区域，不作为置信带。

目标是最小化全天计划购电费：

$$\min C_1 = \sum_{t=1}^{144} p_t g_t. \quad (2)$$

以母线侧收支写能量守恒，将允许的富余供电显式记为未利用电量：

$$g_t + V_t + d_t = L_t + c_t + w_t, \quad g_t, w_t \geq 0. \quad (3)$$

充电损耗发生在进入储能时，放电损耗发生在从储能输出至母线时，因而状态方程为：

$$S_{t+1} = S_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad 1200 \leq S_t \leq 10800. \quad (4)$$

SOC 上下界施加于全部 145 个状态点，状态递推与其他时段约束对应 $t=1, \dots, 144$ 。令每时段最大母线侧充放电量 $M=5000/6$ kWh，用二元变量直接限制互斥：

$$0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad z_t \in \{0,1\}. \quad (5)$$

第一问满足以下初始和终端条件，避免利用电池初始存量净放电造成虚假节省：

$$S_1 = S_{145} = 6000. \quad (6)$$

将上述关系集中写成一般日模型，决策变量为购电、充放电、未利用电量、储电量和状态变量。给定不同供需输入与初末状态条件时，可复用以下结构：

$$\left\{ \begin{array}{ll} \min_{g,c,d,w,S,z} & \sum_{t=1}^{144} p_t g_t \\ \text{s. t.} & g_t + V_t + d_t = L_t + c_t + w_t, \\ & S_{t+1} = S_t + \eta_c c_t - d_t / \eta_d, \\ & 1200 \leq S_t \leq 10800, \quad g_t, w_t \geq 0, \\ & 0 \leq c_t \leq M z_t, \quad 0 \leq d_t \leq M(1 - z_t), \\ & z_t \in \{0,1\}, \quad S_1 = S_{145} = 6000. \end{array} \right. \quad (7)$$

上述模型有 865 个变量，其中 144 个为二元变量。原方案以 SciPy 1.17.1 的 milp 接口调用 HiGHS [2]，相对间隙目标设为 10^{-7} ，每日求解时间上限为 30 秒；超时或无可行解时应停止并报告，而非将空结果视为成功。

5.2 调度结果与指定时段

计算得到全天购电量 59,482.70 kWh、费用 35,126.95 元。购电与储能轨迹见图 2，指定时段购电和四小时充放电分别见表 3、表 4。

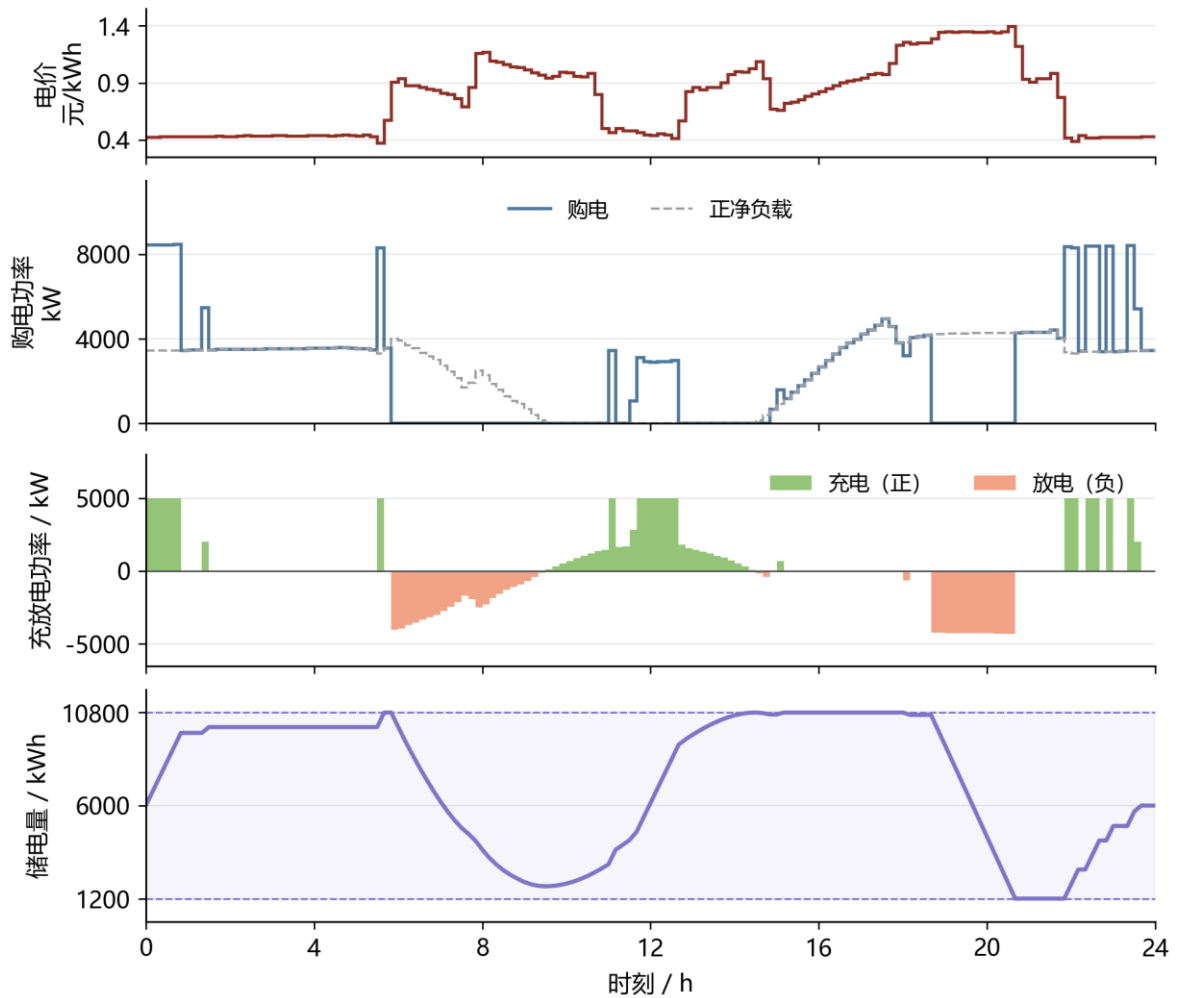


图 2 第一问电价驱动的购电与储能运行

自上而下为电价、购电与正净负载、充放电、SOC。所有阶梯保留十分钟决策，不作平滑。购电超过正净负载的部分用于充电；SOC 阴影为允许范围。

表 3 第一问指定时段购电量及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量 59,482.70 kWh			全天购电费 35,126.95 元		

表中电量单位为 kWh，全天购电费单位为元。

表 4 第一问四小时充放电及日初日末储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	6,365.84
08:00-12:00	4,787.96	1,703.00	12:00-16:00	5,286.04	91.10
16:00-20:00	0.00	5,780.13	20:00-24:00	5,333.33	2,859.87
00:00 储电量	6,000.00		24:00 储电量	6,000.00	

表内所有电量单位为 kWh，充放电量采用母线侧口径。四小时内可先充后放，汇总行两者均正不违反十分钟互斥。图中尖峰主要对应集中充电：22:00—22:10 负载约 3309.39 kW、光伏为零，充电 5000 kW，故购电约 8309.39 kW。相邻时段电价为 0.3859、0.4368 元/kWh，模型分别安排满功率充电与不充电。

当前目标只计购电费，未计充电切换成本或功率爬坡限制，因此低价区间内也可能出现零散充电。这是模型边界，不能由图形尖峰直接判为原始数据异常。固定放电与未用电量、保持费用不变的充电变差最小化诊断未获得实质不同的轨迹，不能据此声称存在平滑且同成本的替代方案。若需要更平稳的实际操作，应另加爬坡约束或费用容差内的次级目标，重新计算并比较成本。

5.3 最优性检验与效率解释

删除整数限制得到线性松弛下界 35,126.94858929 元，与整数解目标之差为 0.00000000 元，求解器相对间隙为零。在本题输入与约束口径下，这支持数值精度内的全局最优性。能量平衡最大残差约 2.27e-13 kWh，SOC 递推与容量、功率边界均通过复核。

以每时段正净负载直接购电构造无储能基线：

$$g_t^{(0)} = \max(L_t - V_t, 0), \quad c_1^{(0)} = \sum_t p_t g_t^{(0)}. \quad (8)$$

该基线费用为 48,052.05 元，主模型节省 26.90%。效率解释的对照见图 3：若总往返效率为 0.9，并对称取两侧效率为平方根 0.9，费用为 33,801.50 元。该试验只改变效率，不能与主实验混为同一组结果。

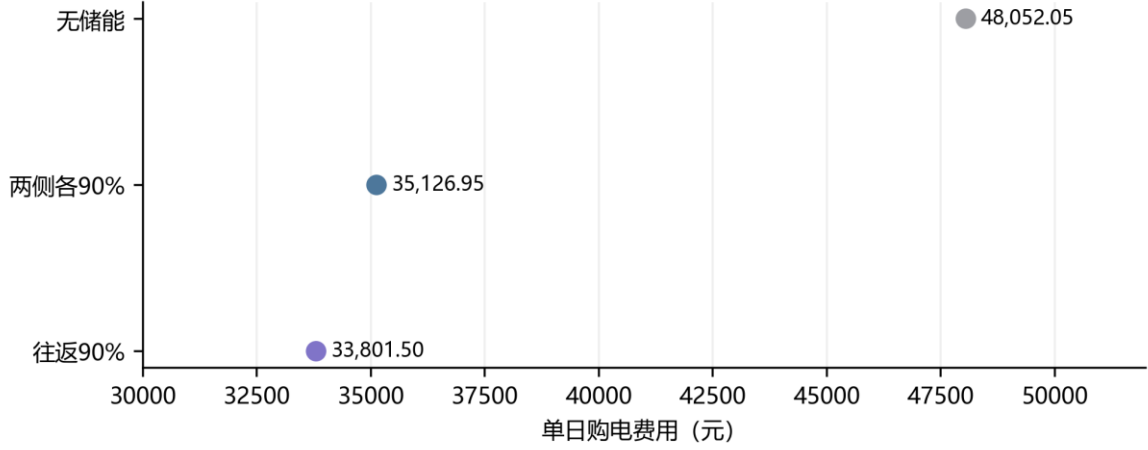


图 3 第一问储能收益与效率解释对照

点图以绝对费用标注。各点分别比较无储能、主效率和往返效率解释；主文及 Excel 均对应“两侧效率各 90%”。

5.4 互斥约束的等价线性化

互斥不能由一般 LP 自动保证，但本题的自由弃电变量允许给出更强的结构结论。令 $\rho = \eta c \eta d \leq 1$ 。对松弛解任一时段，作如下变换：

$$x_t = \min\{c_t, d_t/\rho\}, \quad c'_t = c_t - x_t, \quad d'_t = d_t - \rho x_t, \quad w'_t = w_t + (1 - \rho)x_t. \quad (9)$$

充电量和放电量均不增加，且至少一个变为零。由于 $\eta c x = \rho x / \eta d$ ，储能增量完全不变；由于 $d' - c' - w' = d - c - w$ ，母线平衡也保持不变。购电、紧急购电和所有计划增减量不变，因此本文各类费用均不变。逐槽处理后得到符合互斥约束的物理解：

$$C_{LP}^* \leq C_{MILP}^* \leq C_{\text{transformed LP}} = C_{LP}^*. \quad (10)$$

因此，在本模型条件下，二者最优费用相等。Q1 的变量可从 865 个减少到 721 个，并去掉 144 个二元变量；日内调整模型同理。实现使用 HiGHS 线性规划，求解后显式消除循环并重新核验物理约束，绝不直接把同时充放电的松弛解作为运行方案。

证明依赖允许非负、无上界、无额外费用的未利用电量，以及仅由购电和调整量决定的目标函数。若加入严格弃电限制、禁止弃用已买电量、充放电最短持续时间、切换费用等约束，应重新证明或恢复整数模型。本结论不等于所有储能规划都可删除互斥变量。相同最优费用也不保证求解器返回同一计划；在不确定输入下，不同同价计划仍可能产生不同事后费用。

六 第二问的预测与日前策略

6.1 训练与评价的时间顺序

以 1 月 22—31 日作为验证窗，选择岭回归惩罚与风险余量。2 月 1 日起固定这些选择规则，执行至 12 月 31 日。模型每七天更新一次，最多使用最近 60 个已结束日期；评

价过程中已观察到的数据可以进入后续更新，尚未发生的数据不进入当前决策。这是顺序样本外回测，而不是把全年随机拆分。

一月份共同热身从 1 月 1 日的 6000 kWh 开始；首日无历史时用附件 1 作为冷启动参考，随后不足七天用近三日均值，积累七天后用日 / 周同期。验证期初 SOC 为 6244.256891 kWh，评价期初为 9902.811288 kWh，各候选策略都从同一状态出发。冷启动的附件 1 仅作为明确的初始化假设，不当作全年可提前获得的预报。

6.2 同期基线与岭回归预测

对负载或光伏电量序列 y ，同期基线取昨日与上周同日相同时间槽的均值：

$$\hat{y}_{d,t}^{\text{seasonal}} = \frac{y_{d-1,t} + y_{d-7,t}}{2}. \quad (11)$$

岭回归采用 21 个特征：昨日、前日、上周同日的同槽值，三日和七日同槽均值、七日同槽标准差、昨日全天均值和近三日全天均值共八项；前三阶日内正余弦六项；星期指示七项。日期及时间属于决策时已知的日历信息。标准化均值与尺度仅在当次训练集上估计，常量特征尺度置 1。

$$\min_{\beta, b} \sum_{i \in \mathcal{T}_d} (y_i - b - \tilde{x}_i^T \beta)^2 + \lambda \|\beta\|_2^2. \quad (12)$$

截距不惩罚，负载和光伏分别拟合。按验证 MAE 比较 $\lambda = 1, 10, 100$ ，均选 1；实现采用 NumPy 线性方程求解，与带截距岭目标一致 [3]。从 1 月 15 日开始具备拟合条件，之后每七日重新拟合。预测截断为非负；某槽过去七日光伏全零时，该槽预测置零，日出日落季节变化可能使此规则产生滞后。

评价指标采用功率单位的 MAE 与 RMSE；不使用夜间分母为零的普通 MAPE：

$$\text{MAE} = \frac{1}{n} \sum_i |P_i - \hat{P}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (P_i - \hat{P}_i)^2}. \quad (13)$$

6.3 净负载误差余量与名义优化

定义历史净负载误差及只包含过去信息的误差样本池：

$$\varepsilon_{j,s} = (L_{j,s} - V_{j,s}) - (\hat{L}_{j,s} - \hat{V}_{j,s}). \quad (14)$$

$$\mathcal{E}_{d,t} = \{\varepsilon_{j,s} : \max(2, d-28) \leq j < d, 1 \leq s \leq 144, |s-t| \leq 3\}. \quad (15)$$

日期下标以 1 月 1 日为 $d=1$ ；排除首日冷启动误差，最多池化 28 个已结束日期和目标槽前后三槽。跨日边界不绕回，不足 28 天时仅取已有历史。余量规则为：

$$r_{d,t}(q) = \max\{0, Q_q(\mathcal{E}_{d,t})\}. \quad (16)$$

无余量候选直接设 $r=0$ ，与“取零分位数”区分。对 $q=0.7, 0.8, 0.9$ ，分位数使用样本排序后的线性插值。将预测负载加余量作为名义需求，求解第五节同一物理模型：

$$L_t^{\text{nom}} = \hat{L}_{d,t} + r_{d,t}(q), \quad V_t^{\text{nom}} = \hat{V}_{d,t}, \quad S_{d,145}^{\text{nom}} \geq 6000. \quad (17)$$

70% 分位数不是“全天 70% 概率不缺电”的保证。名义优化只最小化计划费，五倍紧急购电的影响通过验证期对余量方案的筛选间接体现，不能据此称为已精确求解随机规划。

6.4 实时执行与跨日状态

当天普通购电计划固定后，定义当前供需剩余。对非负剩余尽量充电，对缺口尽量放电，并分别受可用空间、储电量和功率限制：

$$u_t = g_t + V_t - L_t. \quad (18)$$

$$c_t = \min \left\{ \max(u_t, 0), M, \frac{10800 - S_t}{\eta_c} \right\}. \quad (19)$$

$$d_t = \min \{ \max(-u_t, 0), M, \eta_d (S_t - 1200) \}. \quad (20)$$

$$e_t = \max(-u_t - d_t, 0), \quad w_t = \max(u_t - c_t, 0). \quad (21)$$

以上规则使同一时段仅有充电或放电，SOC 仍按状态方程更新，实际日末传给次日：

$$S_{d+1,1} = S_{d,145}. \quad (22)$$

所有供电缺口由紧急购电补足。全年评价费用定义为：

$$C_2 = \sum_{d,t} p_t g_{d,t} + 5 \sum_{d,t} p_t e_{d,t}. \quad (23)$$

将第二问策略概括如下。记 Ω 为第一问中除初末等式之外的物理可行域，输入替换为带余量预测，日初状态取真实已知值；G 表示本节的贪心实时平衡规则：

$$\left\{ \begin{array}{l} (g^{\text{plan}}, \dots) \in \arg \min_{(g,c,d,w,S,z) \in \Omega} \sum_t p_t g_t, \\ L^{\text{nom}} = \hat{L} + r(q), \quad V^{\text{nom}} = \hat{V}, \\ S_1^{\text{nom}} = S_1^{\text{actual}}, \quad S_{145}^{\text{nom}} \geq 6000, \\ (c_t, d_t, e_t, w_t) = \mathcal{G}(g_t^{\text{plan}}, L_t, V_t, S_t), \\ C_2 = \sum_{d,t} p_t (g_{d,t}^{\text{plan}} + 5e_{d,t}). \end{array} \right. \quad (24)$$

其中 Ω 中每一时段的平衡使用上式名义输入；最后一行是实际策略评价费用，不是声称日前已知实际 e 的优化目标。余量 q 仅由一月验证费用选择，之后冻结。

实际储能动作可以偏离名义动作，但普通计划购电不能按真实曲线事后改写。这一贪心控制具有可行性与因果性，不保证跨时段的实时经济最优。

七 第二问的结果与检验

7.1 验证期选择与紧急购电权衡

验证结果见表 5 与图 4。各方案使用相同验证初始 SOC，但期末存量不同，因此除费用外同时列出期末储电量，不把存量差异隐藏在成本比较中。

表 5 一月份十天验证结果

预测器	余量	总费/元	紧急费/元	期末 SOC/kWh
同期	无	661,834.09	173,098.38	9,902.81
同期	70%	634,179.58	110,989.47	10,595.98
同期	80%	666,226.97	64,910.51	10,800.00
同期	90%	680,257.72	456.98	10,800.00
岭回归	无	532,888.41	61,836.07	5,924.08
岭回归	70%	509,757.47	8,765.44	6,623.17
岭回归	80%	529,846.45	4.16	7,110.16
岭回归	90%	633,664.05	0.00	9,039.38

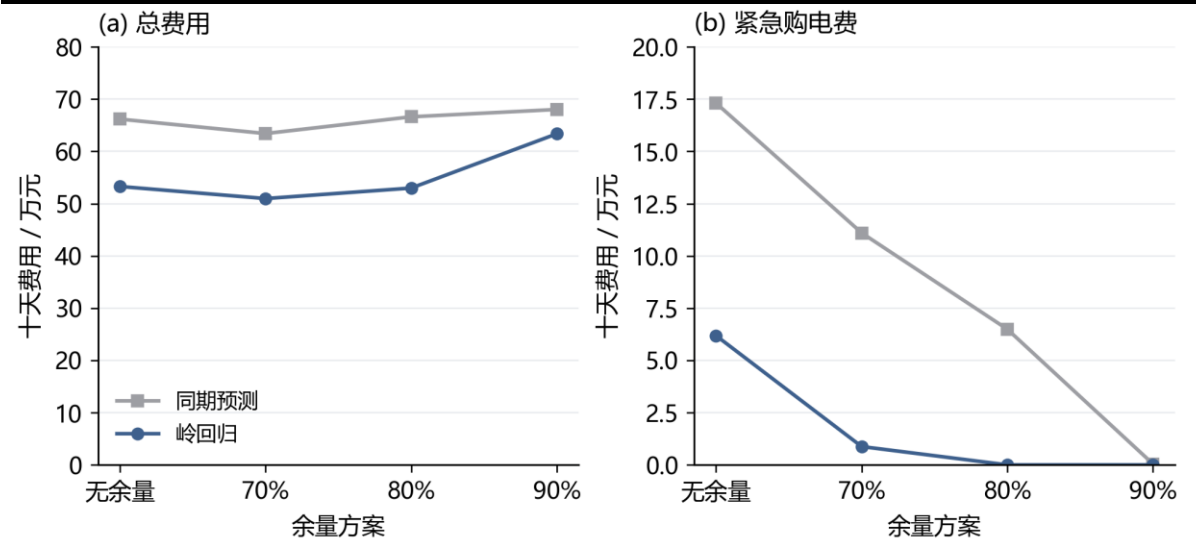


图 4 验证期风险余量与费用的权衡

横轴为四种候选方案，其中“无余量”是独立基线。折线只辅助比较候选值，不表示连续参数的精确响应。

岭回归 70% 余量的验证费为 509,757.47 元，其中紧急费 8,765.44 元；90% 余量紧急费为零，但总费升至 633,664.05 元。两者期末储电量相差约 2416 kWh，故不能完全忽略存量影响；当前目标仍按题面购电费用评价，不另添加缺乏依据的残值收入。

7.2 全期预测误差与季节变化

按表 6 比较 334 天共 48,096 个十分钟区间的功率预测误差，岭回归在负载预测上的改善大于光伏。四个指定日期的曲线对照见图 5，整年误差仍以全期统计为准。

表 6 第二问全期预测误差

预测器	负载 MAE	负载 RMSE	光伏 MAE	光伏 RMSE
同期	372.26	585.91	171.98	343.38
岭回归	179.86	234.59	155.28	306.74

上述误差单位均为 kW，光伏指标包含夜间零值，不能直接等同于白天发电时段的预测精度。

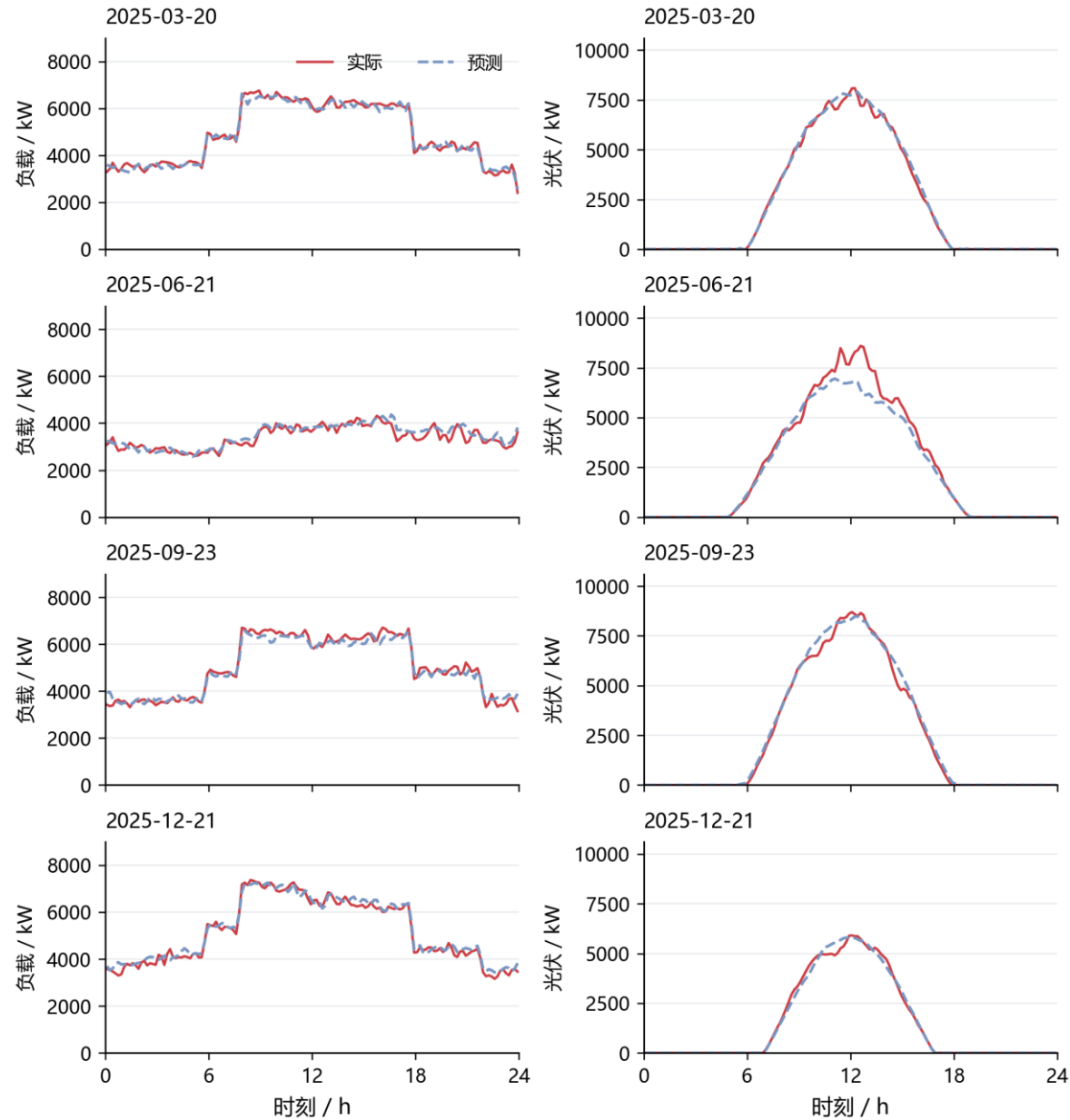


图 5 四个指定日期的负载与光伏日前预测

各列共用纵轴尺度，日期由题目指定。实线为实际观测，虚线为岭回归预测，不加入平滑后虚构的尖峰。

7.3 全期购电费用与运行约束

策略费用分解如图 6，数值及期末存量见表 7。相同初始状态保证比较具有共同起点，实际状态在全年内自行连续演化。

表 7 第二问全期费用及期末储电量

策略	计划费/万元	紧急费/万元	总费/万元	期末 SOC/kWh
同期 无余量	1,199.19	723.59	1,922.78	5,825.99
岭回归 无余量	1,215.92	354.62	1,570.54	5,903.65
同期 70%余量	1,297.86	452.54	1,750.40	6,217.35
岭回归 70%余量	1,293.50	119.76	1,413.26	5,903.65

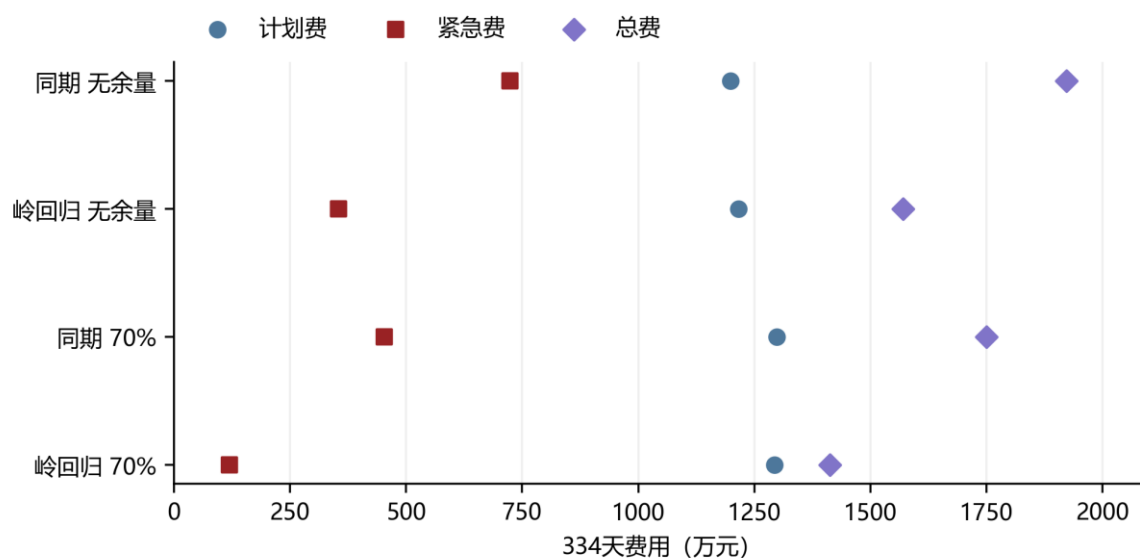


图 6 第二问四组策略的全期费用分解

蓝色为普通计划购电费，深红为紧急购电费；用点的位置分别表示两项及总费用。主策略由一月份验证选择。

主策略总费用 14,132,612.16 元，较同期无余量低 26.50%，较岭回归无余量低 10.01%。后两组期末 SOC 均为 5,903.65 kWh，因此此处的余量增益没有被不同期末存量混淆。

主策略紧急购电累计 197,465.10 kWh，涉及 1132 个十分钟区间，不等同于同样数量的独立事件。未利用电量为 2,211,229.49 kWh，其中可包含未消纳光伏和富余计划购电，不能全部称为弃光。

按月费用见图 7，主策略在 11 个评价月份中有 11 个月的总费用低于同期无余量。月度划分仅用于描述已固定策略的季节表现，未用于重新选超参数；月总费也受到月份天数和负载规模影响，不宜直接将其解读为单位供电效率。

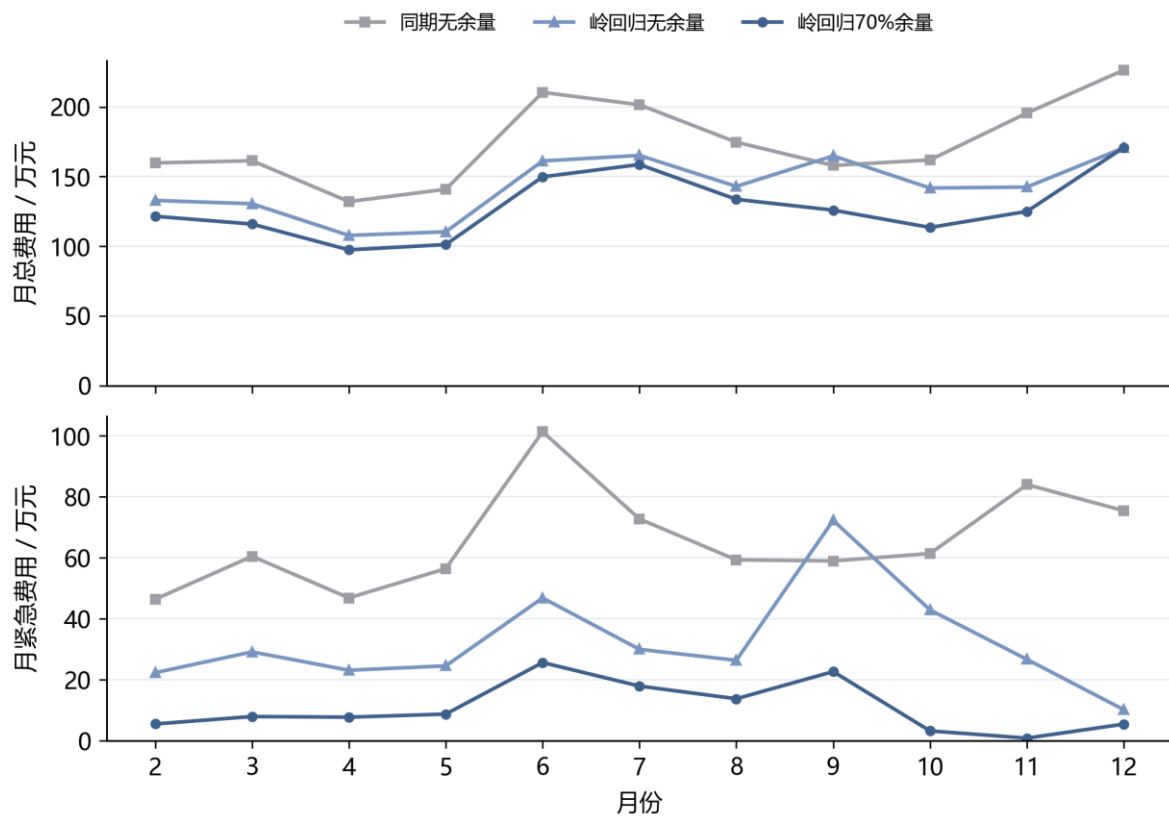


图 7 固定策略的月度总费用与紧急费用

上面板为含紧急费用的月总费，下面板为其中的紧急费用。三条曲线依次分离预测改进和风险余量的作用。

月度预测 MAE 进一步见图 8。两块热力图各用自身的颜色上限，不能直接按跨面板颜色深浅比较负载与光伏误差；应读取格内数值。该图用于观察季节变化，不提供未计算的置信区间。

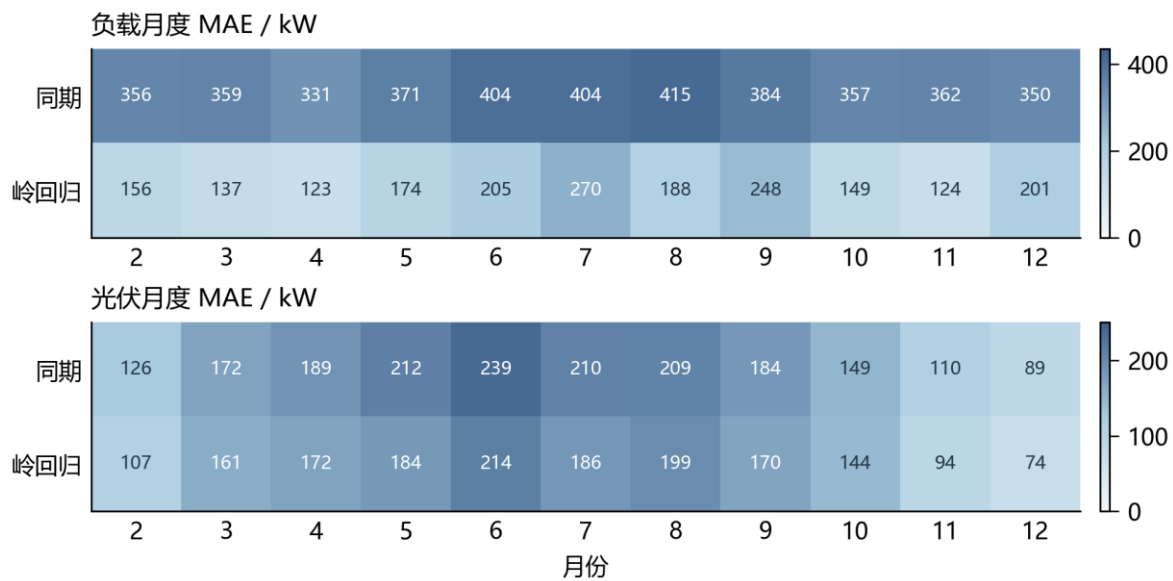


图 8 同期预测与岭回归的月度误差

每格为对应月份全部十分钟区间的 MAE，单位 kW；数值为整数显示，统计文件保留完整精度。

四个指定日期的购电、充放电、日初日末储电量及紧急事件，集中列于附录 A；完整逐日计划分别写入题目指定结果文件。

7.4 日末目标修订的失败证据

名义日末 6000 kWh 是策略设置，而非问题二的硬约束。新增候选比较余量 0、0.5、0.7、0.8、0.9 与名义日末 1200、6000、9000 kWh，共 15 组，均只用一月 22—31 日费用选择。选择到 0.7 和 1200 kWh。但该候选全年费用与库存状况如下。

对应结果见表 8。

表 8 第二问新终端候选与保留方案

方案	总费/元	期末 SOC/kWh
原策略（保留）	14,132,612.16	5,903.65
一月选择的新候选	14,142,959.63	1,311.08

候选全年增加费用 10,347.47 元，并少保留 4,592.57 kWh 库存，不能称为改善。按全期最大固定电价估计补齐差额所需购电的压力金额为 7,119.51 元；它只是存量核算压力分析，不是虚构发生在年末后的实际交易。V2 交付继续采用原策略。本次保留与替换属于同一数据集上的修订选择，不能再把交付组合声称为未经观察的独立测试结果。

八 第三问的日内滚动调整

8.1 新预报的价值与时间对齐

第三问新增的是带有发布时间的光伏信息，而不是未来实际出力。每个更新点应比较新计划可能避免的紧急购电与调整支出，并保护已经执行的交易。负载仍沿用第二问的日前岭回归，日内不使用未来负载真值校正预测；这一设计也使更新时间对照主要反映光伏预报和当前储能状态的作用。滚动优化具有不断用最新可用信息更新未来决策的结构，其思想可参见文献[4]；本文没有照搬该文的潮流或分布式控制模型。

附件 3 含 365 天、每日 4 次、每次 24 个小时预报，共 35,040 个数值。1,095 个空日期位于同日四行预报的后 3 行，只在该结构内补齐。预报发布点与提前量逐行核验，无缺失、负数和非有限数；17,525 个零值保留。预报“1 小时”指发布后一小时，不是当天 01:00。附件 4 的 52,560 个电价亦完整、非负且严格为正，范围 0.0076—1.7936 元/kWh。两附件均不做删点、缩尾或平滑观测。

设发布时刻为 τ ，小时预报端点为 $H\tau, h, h=1, \dots, 24$ 。发布时间处缺少零小时端点，采用刚结束的十分钟实际光伏均值作为边界代理，年初无历史时取零；该代理只用于预报曲线首段。以相邻整点之间的线性函数插值，并对每个十分钟区间作梯形积分：

$$\hat{V}_{\tau,t} = \int_{a_t}^{b_t} \tilde{P}_{\tau}(u) du = \frac{\tilde{P}_{\tau}(a_t) + \tilde{P}_{\tau}(b_t)}{2} \Delta t. \quad (25)$$

此过程是预报分辨率转换，不是修改实际数据。实际电量仍按附件 2 原值乘 1/6 小时。当前实现每次只优化至本日 24:00，预报中次日部分暂不用于跨日承诺，避免题目未规定的提前购入次日电量。图 9 展示 6 月 21 日的实际与不同版本预报。

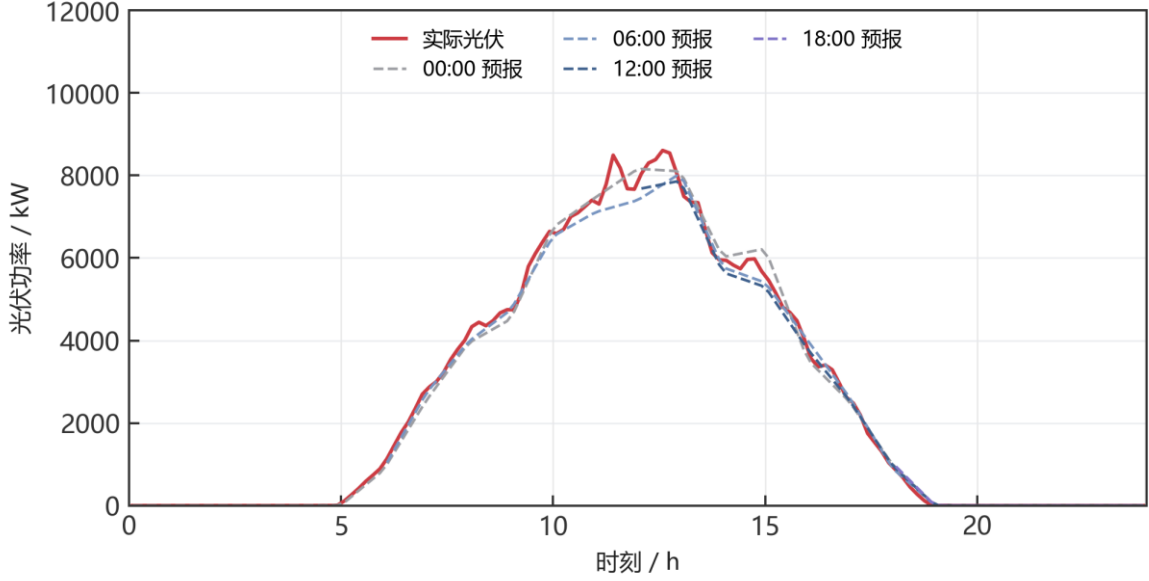


图 9 同一日期不同发布时刻的光伏预报

各版本只从其发布时刻向后显示。新增预报不保证每个目标时段更接近实际；不能把 18:00 版本提前用于午夜决策。

8.2 多次调整的费用账本

题面给出减购 50% 违约电价和增购 1.5 倍价格，但没有完整说明取消部分是否退回原款。主模型取取消原价退款并扣 50% 违约费，即净回收 0.5 倍原价；另将“原款不退且额外支付 50% 违约费”单独重新优化作敏感性。该分歧必须体现在假设和结果中，不能默认为唯一解释。

令 g^0 为午夜计划， g^k 为第 k 次更新后的计划。每次调整对最近版本定义非负增购 a^+ 和减购 a^- 。已执行区间不允许进入调整集合，未来计划保持非负：

$$g_t^k = g_t^{k-1} + a_{k,t}^+ - a_{k,t}^-, \quad a_{k,t}^\pm \geq 0, \quad 0 \leq a_{k,t}^- \leq g_t^{k-1}. \quad (26)$$

单次同一区间不应同时增减。因为价格为正，同时增加和减少同量会额外支付正费用，可从最优解中消去；程序仍检查实际解没有这种循环。完整账本为：

$$C_3 = \sum_t p_t g_t^0 + \sum_k \sum_{t \in T_k} p_t (1.5a_{k,t}^+ - 0.5a_{k,t}^-) + 5 \sum_t p_t e_t. \quad (27)$$

对于多轮操作，新增单位支付 1.5 倍，随后取消也只净返还 0.5 倍，绝不返还先前支付的全部 1.5 倍；因而先增加再取消的往返费用为一倍价格，不产生虚假套利。总费用拆成午夜原始计划、历次增购、历次减购净退款、实际紧急购电四部分。另一个无退款解释只把负 0.5 系数改为正 0.5，并重新求解整个策略。

8.3 滚动优化与余量校准

每次重新优化以当前真实已知储电量为初值，沿用物理可行域和名义日末 6000 kWh 下限。00:00 以全部计划费用为目标；其余时刻已付款为常数，仅最小化尚可改变部分的调整费用。余量按对应发布版本、同一目标钟点此前 28 天净负载误差计算，并池化相邻三槽；不足 28 天使用已有历史。零余量单独作为候选。

$$\left\{ \begin{array}{l} \min_{g,c,d,w,S,z,a^+,a^-} \quad \sum_{t \in \mathcal{T}_k} \hat{p}_{k,t} (1.5a_{k,t}^+ - 0.5a_{k,t}^-) \\ \text{s. t.} \quad g_t = g_t^{k-1} + a_{k,t}^+ - a_{k,t}^-, \\ \quad g_t + \hat{v}_{k,t} + d_t = \hat{L}_t + r_{k,t} + c_t + w_t, \\ \quad (g, c, d, w, S, z) \in \Omega, \\ \quad S_{\tau_k} = S_{\tau_k}^{\text{actual}}, \quad S_{145} \geq 6000, \\ \quad a_{k,t}^+ \geq 0, \quad 0 \leq a_{k,t}^- \leq g_t^{k-1}. \end{array} \right. \quad (28)$$

Ω 沿用第一问中状态递推、所有状态点储能边界、母线侧功率限制和二元互斥，去掉原初末相等式。第三问 $\hat{p}$ 就是附件 1 的已知固定价格。求解后只按实际逐十分钟执行到下一更新点，未来轨迹随后可重新计算。没有更新的组保持已承诺计划，实际电池仍按即时余缺充放电并在不足时紧急补购。该控制规则保证供电可行，但不保证随机控制意义下最优。

一月 22—31 日新增比较余量 0、0.5、0.7，负载校正强度 0、0.5、1，以及 12/18 点或 6/12/18 点更新，共 18 组，名义日末下限均为 6000 kWh。此前原方案只比较 0、0.7、0.9 三个余量，因此没有覆盖适中的 0.5。新增参数的价值来自更贴近调整制度下的欠购与多购权衡，而非使用更复杂的预测器。

校正候选只取更新时刻之前最近 18 个已完成区间的负载预测残差均值，并随未来提前量按 36 个时段的尺度指数衰减。午夜校正为零，更新点当前区间和未来实际值均不可进入：

$$\bar{\epsilon}_\tau = \frac{1}{m} \sum_{i=\max(0, \tau-18)}^{\tau-1} (L_i - \hat{L}_i), \quad \hat{L}'_{\tau,t} = \max\{0, \hat{L}_t + \beta \bar{\epsilon}_\tau \exp[-(t - \tau + 1/2)/36]\}. \quad (29)$$

残差池对相同发布版本与相同校正规则建立，使用此前 28 个完整日期及目标前后三槽。过去时段绝不按后来发布的信息回填。选择仍以验证实际费用为准，结果为 $q=0.5$ 、 $\beta=0$ 、12/18 点更新。额外负载校正验证期未得到支持，故主模型保留 $\beta=0$ 。

8.4 成本改善及来源

对应结果见表 9。

表 9 第三问费用账本与运行量

指标	原策略	V2
午夜计划费/元	12,282,761.23	12,521,335.94
增购费/元	720,056.59	503,962.50
减购净费用/元	-145,798.31	-148,871.09
紧急费/元	1,231,545.41	888,890.35
总费/元	14,088,564.93	13,765,317.70
紧急购电/kWh	230,671.95	166,980.76
未利用电量/kWh	1,375,609.93	1,485,682.88
期末 SOC/kWh	5,903.65	5,903.65

V2 节省 323,247.23 元，降幅 2.29%。午夜多购增加常规支出，同时减少昂贵日内增购和五倍紧急购电，形成净节省。但未利用电量有所增加，因此不应把成本改善写成所有运行指标同时改善。两种方案均从 9902.811288 kWh 开始，期末均为 5,903.65 kWh，比较没有被期末存量差异混淆。

图 10 进一步给出两项滚动策略的累计节省路径。

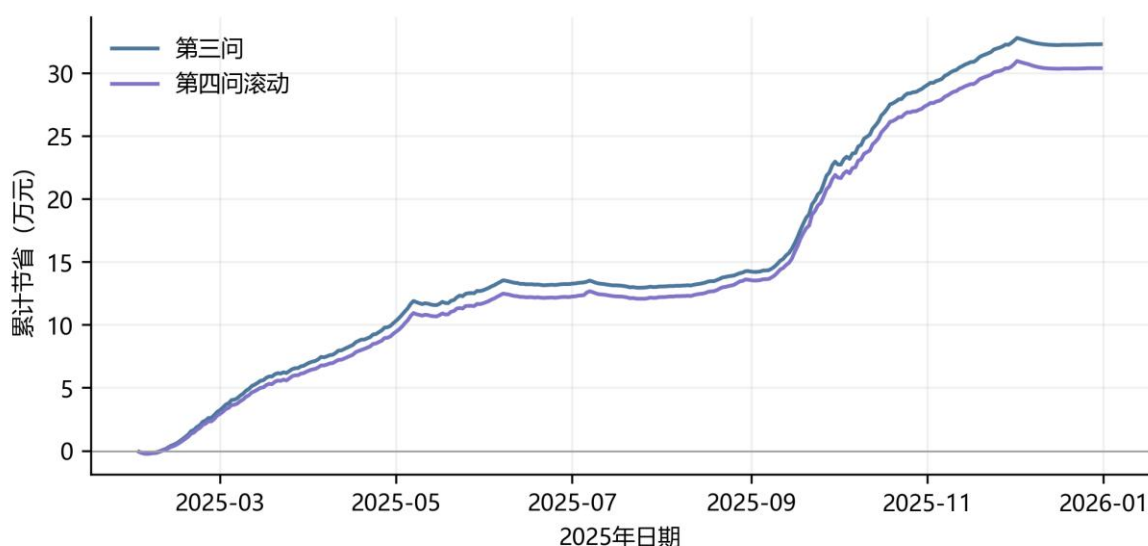


图 10 第三问与第四问滚动策略相对各自原策略的累计节省；曲线上升表示累计节省扩大，局部回落表示该时段改进方案反而更贵。全部日期连续纳入，不挑选有利月份。

原稿八种更新组合的全年结果仍保留，但属于原余量设置下的比较；不能当作新 $q=0.5$ 的八组合结果。原策略相对只午夜更新具有收益，这支持研究日内信息价值。新策略只在事前列明的两个更新集合内选择，不能宣称 12/18 点已在所有可能更新时间中全局最优。

8.5 消融与结算敏感性

对应结果见表 10。

表 10 第三问受控对照

方案	总费/元	紧急购电/kWh
LP、原 $q=0$ 规则	14,088,571.35	230,671.95
主方案 $q=0.5$ 、 $\beta=0$	13,765,317.70	166,980.76
相同 q 与时刻、 $\beta=1$ 消融	13,714,319.42	151,040.79
不退款另罚 50%，重新求解	13,825,450.94	165,118.76

仅改为 LP 的全年费用与原 MILP 结果有数元差异，不是节省的主要来源；相同名义最优值不等于浮点运算、同价计划与跨日执行路径逐位相同。 $\beta=1$ 消融的全年费用更低，但这是事后观察，不能覆盖一月选择。它说明负载校正值得后续独立季节验证，不能据此将其提升为已验证主策略。不退款敏感性保持参数不变并重新优化，实际计划也随之变化，差额不能解释为单纯退款金额。

九 第四问的电价预测与两类调度

9.1 波动价格的信息与结算

附件 4 每个供电区间只有一个实际价格，没有提供同一交付区间在不同交易时点的报价面板。主模型将未来价格视为未知，历史值仅在区间结束后可用；预测价格进入优化，实际交付区间价格乘对应系数用于回测结算。这是当前数据能够支持的明确假设，而非声称全年价格已在午夜公开。图 11 给出实际价格的全年变化。

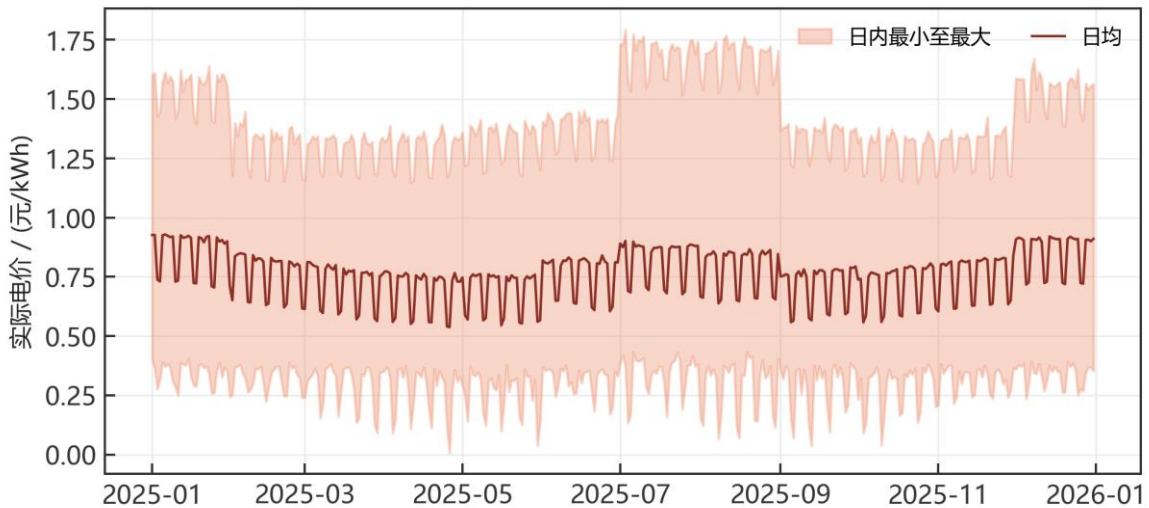


图 11 全年实际电价的日内范围与日均值

阴影为每天实际最小至最大值，不是置信区间；没有删除价格峰谷。价格低至 0.0076 元/kWh 仍保留，不以 MAPE 作主指标。

9.2 公平预测接口与时间切分

每个起点 τ 直接预测未来 144 个十分钟价格。输入为目标钟点前一天和前一周的两条历史曲线，以及目标时刻、星期的正余弦编码；在各起点这些信息全部已知。同期、

岭回归与 GRU 的信息来源相同，表示方式不同。前一天的末项最大为 τ 前一个已经结束的区间，避免把多步预测的“最近滞后”错误取成未来真值。

$$\hat{p}_{\tau,j}^{\text{seasonal}} = \frac{p_{\tau+j-144} + p_{\tau+j-1008}}{2}, \quad j = 0, \dots, 143. \quad (30)$$

岭回归把两条历史价格、日历、两者差和乘积及各历史曲线均值和标准差组合为 12 维特征，按未来目标逐行拟合。标准化参数仅由训练窗口估计，正则强度在 1、10、100 中依据一月验证 MAE 选择，选得 1。序列窗口每 6 小时移动；窗口重叠不能视为完全独立样本。全部训练目标必须在拟合时刻前结束，不能仅检查窗口起点。

模型于 1 月 15 日、22 日初始化，此后每月月初从最近 60 天历史重新拟合。GRU 每次把最近 3 天完整目标窗口作为内部早停验证，其他可用窗口训练，跨越分界的目标窗口剔除。内部验证只服务该次训练；一月 22—31 日外层验证用于比较方案；2—12 月结果用于顺序样本外评价，不能反过来选择结构、种子或主方案。

9.3 残差 GRU 的训练

为避免小样本网络从零重建已有日周期，先以同期曲线作为基准，再用单层 GRU 学习偏差。输入维度 6，隐藏维度 32，每个位置输出一个价格残差，整体一次得到 144 步，不使用未来标签进行递归输入。门控关系采用 PyTorch 的实现约定[5]：

$$r_j = \sigma(W_{ir}x_j + b_{ir} + W_{hr}h_{j-1} + b_{hr}), \quad z_j = \sigma(W_{iz}x_j + b_{iz} + W_{hz}h_{j-1} + b_{hz}). \quad (31)$$

$$n_j = \tanh(W_{in}x_j + b_{in} + r_j \odot (W_{hn}h_{j-1} + b_{hn})), \quad h_j = (1 - z_j) \odot n_j + z_j \odot h_{j-1}. \quad (32)$$

$$\hat{p}_{\tau,j} = \max\{0.001, \hat{p}_{\tau,j}^{\text{seasonal}} + s_y(w^T h_j + b)\}. \quad (33)$$

历史价格先按训练集尺度标准化， s_y 为相应标准差。输出头初始为零，初始预测因此等于同期基线。训练采用平滑 L1 损失、Adam 学习率 0.003、批大小 32、梯度范数上限 1，每次最多 50 轮；内部验证 MAE 连续 6 轮无改善则停止并恢复最好权重。价格下限 0.001 只作用于预测输出，且低于本数据实际最小价格，不属于清洗实际值。

固定种子 17、42、2026；每次重训使用种子与拟合日索引之和，三个模型分别保存，再以等权均值构成预先定义的 GRU 集成。所有种子均进入结果表，不挑最好的一次替代其他结果。图 12 展示 6 月 1 日重训的内部验证收敛，仅代表该次训练，不能冒充全年的验证曲线。

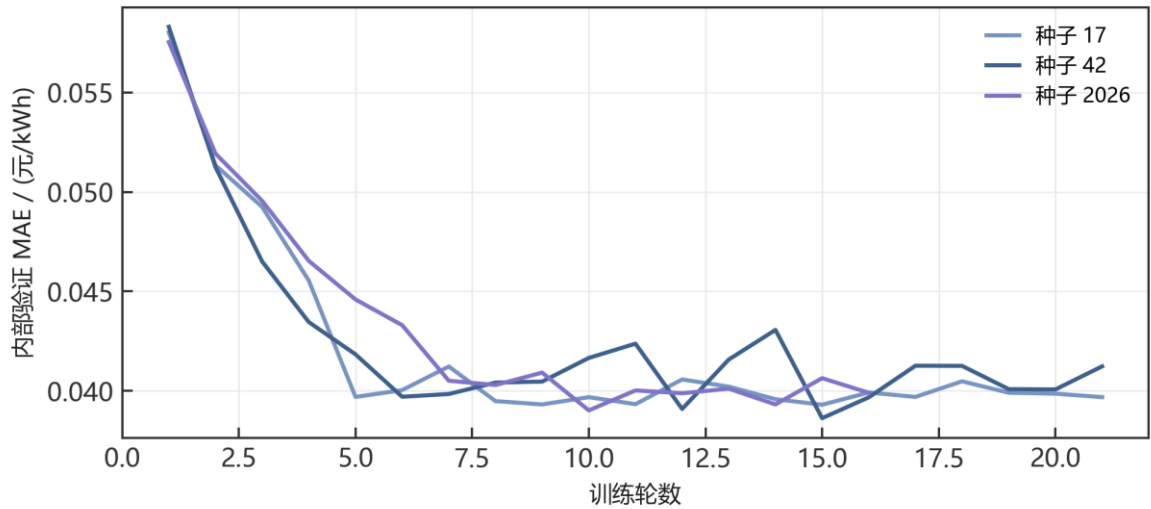


图 12 GRU 三个种子的内部验证过程

曲线在实际早停轮数结束；各次模型均恢复内部验证最优权重。其他月份的完整训练记录保留在支撑材料中。

9.4 按发布时间核对预测误差

全年价误差按发布时间拆分，避免把 Q4-2 实际只用午夜预报的情形与四次发布混合评价。同一目标在不同起点重复出现，不能作为独立重复观测。以下是未更改的已保存价格预测，在原始实际价格上重新计算的误差；本轮未重新训练 GRU，原 39 次训练与三种种子权重均保留。

对应结果见表 11。

表 11 各发布起点价格预测误差

预测器	发布时刻	MAE/元每 kWh	RMSE/元每 kWh	配对数
seasonal	00:00	0.05527	0.07554	48096
seasonal	06:00	0.06117	0.08282	36072
seasonal	12:00	0.05761	0.07870	24048
seasonal	18:00	0.04499	0.06033	12024
ridge	00:00	0.04567	0.06212	48096
ridge	06:00	0.04938	0.06717	36072
ridge	12:00	0.04928	0.06763	24048
ridge	18:00	0.04566	0.06464	12024
gru_mean	00:00	0.04253	0.05726	48096
gru_mean	06:00	0.04603	0.06198	36072
gru_mean	12:00	0.04448	0.06097	24048
gru_mean	18:00	0.03941	0.05546	12024

9.5 两类策略与候选保留原则

日前策略的低终端候选同时改变了储能目标。新候选的价格预测器只根据一月费用在同期、岭回归与 GRU 集成中选择；全年对照未显示其优于原日前方案。因此正式结果 4-2 保留原 70%余量、6000 kWh 名义下限及岭回归价格预测，不用事后最小费用的预测器替换。滚动策略采用 $q=0.5$ 、 $\beta=0$ 、12/18 点更新，同期价格预测仍是一月选择结果。

对应结果见表 12。

表 12 第四问交付与试验结果

方案	总费/元	期末 SOC/kWh
日前原策略（交付保留）	14,880,535.05	5,939.92
日前一月选择的新候选（未采用）	14,887,016.72	1,311.08
滚动原策略	14,790,501.42	5,952.82
滚动 V2（交付采用）	14,486,341.42	5,952.82

对应结果见表 13。

表 13 同一新滚动模型中的价格预测对照

预测器	总费/元	紧急费/元
seasonal	14,486,341.42	877,626.11
ridge	14,485,242.95	863,094.25
gru_mean	14,477,646.38	861,918.33

滚动 V2 比原策略节省 304,160.00 元，降幅 2.06%，期末 SOC 相同。GRU 集成的全期误差与新滚动费用均有优势，但不能因已经看到全年结果就取代一月选出的同期主模型。网络预测、风险余量和调整制度共同决定经济效果，单一平均误差不足以选购电策略。

十 综合核验与稳健性

10.1 原始数据与费用复算

四个输入附件与题面指纹逐一比对；原始时间与单位规范化后，逐时实际负载、光伏和价格与保存轨迹对照，未发现数值篡改或主费用算错。原稿 4 组 Q2 与 21 组 Q3/Q4 轨迹重新通过核验。本轮另对 12 组 334 天新轨迹检查供需平衡、SOC 递推与跨日连续、容量与功率上限、互斥和全部费用分项；对保存的午夜计划逐版本重建增减及最终计划。每次求解状态与耗时单独保存，失败结果不得进入汇总。

第一问 LP 与 MILP 在原始输入及含光伏富余的随机小实例中交叉比较，并对退费与不退费两种修订目标比较最优费用。前缀扰动测试改变当前及未来输入，确认此前负载校正和风险余量不变。实际执行允许区间内即时平衡的近似仍保留，不能把它说成区间开始时知道完整实际平均值。

10.2 配对日费用与时间依赖

将原策略与候选在同一天的费用差配对，按 7、14、28 天循环区块重采样，每种长度 2000 次，考察结论对时间相关性的敏感性。下面区间是本年度日差序列的经验不确定性描述，不是不同年份的可靠覆盖保证。方法仍依赖区块可交换近似；季节非平稳性与模型修订后的选择偏差不能被 Bootstrap 消除。

对应结果见表 14。

表 14 区块重采样下的全年节省区间

问题	区块/天	2.5%/元	97.5%/元
q3	7	224,299.28	434,641.36
q3	14	195,472.59	463,325.38
q3	28	167,271.07	495,927.99
q4-3	7	214,698.67	405,955.00
q4-3	14	184,778.68	439,933.75
q4-3	28	160,265.81	474,847.76

对应结果见表 15。

表 15 各月滚动改进节省

月份	第三问/元	第四问滚动/元
2025-02	31,004.43	28,078.75
2025-03	38,074.21	35,036.63
2025-04	32,458.93	30,038.92
2025-05	26,062.81	24,077.61
2025-06	5,022.58	5,051.48
2025-07	-1,945.48	-50.54
2025-08	12,288.14	13,846.58
2025-09	86,983.91	83,229.08
2025-10	59,552.90	54,498.80
2025-11	34,749.76	31,622.03
2025-12	-1,004.96	-1,269.34

第三问与第四问滚动的各长度区块区间均为正，支持本数据内费用改善的稳定性；仍需新的季节或独立年度数据检验。第二问与第四问日前候选的区间跨越零且库存更少，不支持宣布其优于原模型。

10.3 模型边界与复现限制

时间按右端点解释、两侧 90%效率、逐版本净退款 50%、波动价按交付区间结算，均为明确工作口径。若官方进一步明确其它解释，需要全量重算。只优化到本日 24:00 没有充分利用次日预报，未来可研究跨日价值函数，但不能提前把未知次日实际曲线加入决策。

本轮没有凭观察峰值就删数，没有新增虚构气象或运行数据，也没有声称全局随机最优或全年零紧急购电。线性化减少的是计算复杂度；风险校准改善的是本数据中的决策费用，两种贡献应分开。低终端目标与简单负载校正的失败或不一致结果表明，单一一月窗口对季节泛化仍有限。

十一 结论

四问共用能量、时标、效率与结算约定。第一问给出可证明等价的 LP 求解形式并维持原最优费用；第二问保留原日前策略，避免把短窗内的库存消耗误当作全年改善。第三问新增适中风险分位数，降低昂贵补购与日内增购；第四问在波动价格下复现该滚动收益。完整指定日表和五份结果工作簿补齐第四问原稿展示不足。所有采用与未采用的候选均保留，交付属于可复算的 Version 2 研究稿。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题: 微网与外部电网电力调控策略. 用户提供题面与附件.
- [2] SciPy Developers. `scipy.optimize.milp`; `scipy.optimize.linprog`. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>. 访问日期 2026-09-11. 本文计算使用 SciPy 1.17.1.
- [3] scikit-learn Developers. Ridge. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html. 访问日期 2026-09-11. 本文以 NumPy 实现带截距岭目标.
- [4] Morstyn T, Hredzak B, Aguilera R P, Agelidis V G. Model Predictive Control for Distributed Microgrid Battery Energy Storage Systems. *IEEE Transactions on Control Systems Technology*, 2018, 26(3):1107–1114. DOI:10.1109/TCST.2017.2699159.
- [5] PyTorch Contributors. GRU. <https://docs.pytorch.org/docs/2.14/generated/torch.nn.GRU.html>. 原试验使用 CPU PyTorch 2.14, V2 保留原模型权重与预测.

附录 A 四问指定日期完整结果

电量单位为 kWh，费用单位为元。普通购电与紧急购电分列。滚动问题的指定时段和全天购电量指最终有效普通计划；午夜计划及每次更新完整保存在 Excel 中。所有表由实际轨迹自动生成。

A.1 问题 1

对应结果见表 16。

表 16 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量 59,482.70 kWh			全天购电费 35,126.95 元		

对应结果见表 17。

表 17 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	6,365.84
08:00-12:00	4,787.96	1,703.00	12:00-16:00	5,286.04	91.10
16:00-20:00	0.00	5,780.13	20:00-24:00	5,333.33	2,859.87
00:00 储电	6,000.00		24:00 储电	6,000.00	

A.2 问题 2

对应结果见表 18。

表 18 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	720.76	20:00-20:10	0.00
全天购电量 68,036.47 kWh			全天购电费 41,276.85 元		

对应结果见表 19。

表 19 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,071.86	209.79	04:00-08:00	1,038.99	5,515.86
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	266.98	5,659.90	20:00-24:00	5,687.88	2,954.47
00:00 储电	6,617.05		24:00 储电	6,376.92	

对应结果见表 20。

表 20 紧急购电

时段	电量
无	0.00

对应结果见表 21。

表 21 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,315.40 kWh			全天购电费 21,120.39 元		

对应结果见表 22。

表 22 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	526.76	524.94	04:00-08:00	413.77	4,402.74
08:00-12:00	9,426.35	0.00	12:00-16:00	0.00	0.00
16:00-20:00	151.96	5,493.83	20:00-24:00	6,132.83	2,497.81
00:00 储电	6,945.01		24:00 储电	7,576.71	

对应结果见表 23。

表 23 紧急购电

时段	电量
无	0.00

对应结果见表 24。

表 24 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	788.73	20:00-20:10	0.00
全天购电量 71,561.09 kWh			全天购电费 44,415.98 元		

对应结果见表 25。

表 25 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,252.67	0.00	04:00-08:00	218.14	5,830.96
08:00-12:00	5,558.85	1,896.62	12:00-16:00	4,033.40	575.17
16:00-20:00	231.66	5,642.21	20:00-24:00	6,050.86	2,841.96
00:00 储电	6,072.60		24:00 储电	6,631.50	

对应结果见表 26。

表 26 紧急购电

时段	电量
20:20-20:30	0.33

对应结果见表 27。

表 27 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,145.51 kWh			全天购电费 62,434.14 元		

对应结果见表 28。

表 28 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,629.25	143.74	04:00-08:00	1,398.62	1,538.91
08:00-12:00	5,704.82	7,168.67	12:00-16:00	6,940.07	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,695.76	2,842.53
00:00 储电	6,318.60		24:00 储电	6,443.34	

对应结果见表 29。

表 29 紧急购电

时段	电量
07:50-08:00	23.78

A.3 问题 3

对应结果见表 30。

表 30 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	518.93	14:00-14:10	0.00
16:00-16:10	507.43	18:00-18:10	714.07	20:00-20:10	0.00
全天购电量 65,574.46 kWh			全天购电费 46,485.41 元		

对应结果见表 31。

表 31 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,249.82	348.06	04:00-08:00	877.38	6,637.46
08:00-12:00	5,422.50	1,763.71	12:00-16:00	4,951.55	394.86
16:00-20:00	231.00	5,237.19	20:00-24:00	5,562.08	2,970.81
00:00 储电	6,306.89		24:00 储电	6,191.68	

对应结果见表 32。

表 32 紧急购电

时段	电量
08:40-10:00	1,013.66

对应结果见表 33。

表 33 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	117.96	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,802.44 kWh		全天购电费 19,766.30 元			

对应结果见表 34。

表 34 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	275.76	271.67	04:00-08:00	436.62	4,038.73
08:00-12:00	9,885.67	0.00	12:00-16:00	0.00	0.00
16:00-20:00	57.62	5,815.22	20:00-24:00	5,796.74	2,550.02
00:00 储电	6,051.09		24:00 储电	6,774.21	

对应结果见表 35。

表 35 紧急购电

时段	电量
无	0.00

对应结果见表 36。

表 36 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	518.73	18:00-18:10	781.45	20:00-20:10	0.00
全天购电量 68,447.36 kWh		全天购电费 44,018.76 元			

对应结果见表 37。

表 37 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,292.60	16.12	04:00-08:00	469.64	6,419.11
08:00-12:00	5,753.28	1,896.62	12:00-16:00	4,814.88	515.80
16:00-20:00	103.68	5,866.55	20:00-24:00	5,726.09	2,705.70
00:00 储电	5,736.51		24:00 储电	6,325.22	

对应结果见表 38。

表 38 紧急购电

时段	电量
20:20-20:40	18.88
20:50-21:10	61.36
21:20-21:40	31.77

对应结果见表 39。

表 39 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,848.10 kWh			全天购电费 61,614.26 元		

对应结果见表 40。

表 40 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,657.30	179.36	04:00-08:00	1,565.97	1,572.10
08:00-12:00	5,758.46	7,184.95	12:00-16:00	7,045.59	2,353.00
16:00-20:00	57.98	5,766.67	20:00-24:00	5,676.93	2,842.53
00:00 储电	6,219.20		24:00 储电	6,395.64	

对应结果见表 41。

表 41 紧急购电

时段	电量
07:50-08:00	34.79

A.4-2 问题 4-2

对应结果见表 42。

表 42 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	310.56	20:00-20:10	756.24
全天购电量 67,791.08 kWh			全天购电费 42,355.14 元		

对应结果见表 43。

表 43 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,655.77	152.63	04:00-08:00	1,195.22	5,572.84
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	214.58	7,108.02	20:00-24:00	5,717.23	1,458.76
00:00 储电	6,850.72		24:00 储电	6,409.05	

对应结果见表 44。

表 44 紧急购电

时段	电量
无	0.00

对应结果见表 45。

表 45 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,422.89 kWh			全天购电费 18,518.97 元		

对应结果见表 46。

表 46 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	438.52	3,694.59	04:00-08:00	1,239.76	1,877.96
08:00-12:00	9,421.97	0.00	12:00-16:00	0.00	0.00
16:00-20:00	163.47	5,505.34	20:00-24:00	6,196.26	2,497.81
00:00 储电	7,001.50		24:00 储电	7,631.37	

对应结果见表 47。

表 47 紧急购电

时段	电量
无	0.00

对应结果见表 48。

表 48 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	416.70	20:00-20:10	0.00
全天购电量 71,436.45 kWh			全天购电费 46,451.24 元		

对应结果见表 49。

表 49 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,585.50	0.00	04:00-08:00	760.67	5,830.96
08:00-12:00	5,687.53	1,896.62	12:00-16:00	3,858.02	528.46
16:00-20:00	253.12	6,023.40	20:00-24:00	6,035.33	2,472.31
00:00 储电	6,184.77		24:00 储电	6,633.87	

对应结果见表 50。

表 50 紧急购电

时段	电量
20:10-20:20	10.23

对应结果见表 51。

表 51 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,314.24 kWh			全天购电费 73,263.64 元		

对应结果见表 52。

表 52 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,056.07	143.74	04:00-08:00	1,743.90	2,155.89
08:00-12:00	5,755.95	7,219.80	12:00-16:00	6,952.06	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,755.02	2,845.49
00:00 储电	6,309.24		24:00 储电	6,493.37	

对应结果见表 53。

表 53 紧急购电

时段	电量
07:50-08:00	23.78

A.4-3 问题 4-3

对应结果见表 54。

表 54 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	518.93	14:00-14:10	0.00
16:00-16:10	507.43	18:00-18:10	714.07	20:00-20:10	741.44
全天购电量 65,384.55 kWh			全天购电费 46,964.57 元		

对应结果见表 55。

表 55 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,264.78	249.34	04:00-08:00	1,707.22	6,637.46
08:00-12:00	4,898.64	1,890.35	12:00-16:00	5,475.40	394.86
16:00-20:00	215.46	6,655.69	20:00-24:00	5,594.46	1,541.15
00:00 储电	6,477.58		24:00 储电	6,219.23	

对应结果见表 56。

表 56 紧急购电

时段	电量
08:40-10:00	887.03

对应结果见表 57。

表 57 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	117.96	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,875.45 kWh			全天购电费 17,386.19 元		

对应结果见表 58。

表 58 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	248.99	3,229.59	04:00-08:00	1,265.39	1,786.95
08:00-12:00	9,917.23	0.00	12:00-16:00	0.00	0.00
16:00-20:00	57.62	5,815.22	20:00-24:00	5,858.38	2,548.85
00:00 储电	6,085.48		24:00 储电	6,831.00	

对应结果见表 59。

表 59 紧急购电

时段	电量
无	0.00

对应结果见表 60。

表 60 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	261.91	14:00-14:10	0.00
16:00-16:10	518.73	18:00-18:10	536.06	20:00-20:10	0.00
全天购电量 68,344.63 kWh			全天购电费 46,150.40 元		

对应结果见表 61。

表 61 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,679.06	22.22	04:00-08:00	1,100.85	6,419.11
08:00-12:00	5,846.61	1,896.62	12:00-16:00	4,673.65	467.89
16:00-20:00	96.43	6,092.82	20:00-24:00	5,575.37	2,482.67
00:00 储电	5,727.38		24:00 储电	6,189.57	

对应结果见表 62。

表 62 紧急购电

时段	电量
18:20-18:30	1.08
20:10-20:20	10.23
20:30-20:40	16.63
20:50-21:10	61.36
21:20-21:40	31.77

对应结果见表 63。

表 63 指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,797.09 kWh		全天购电费 72,212.36 元			

对应结果见表 64。

表 64 储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,169.62	178.51	04:00-08:00	232.98	904.57
08:00-12:00	5,772.83	7,199.32	12:00-16:00	7,536.30	2,747.75
16:00-20:00	57.98	5,766.67	20:00-24:00	5,708.45	2,845.49
00:00 储电	6,215.16		24:00 储电	6,420.72	

对应结果见表 65。

表 65 紧急购电

时段	电量
07:50-08:00	34.79

附录 B 可运行支撑材料

五份结果文件位于 `artifacts/v2`；`src/v2_model.py` 为线性规划及因果调度，`src/v2_experiments.py` 生成全部候选，`src/v2_verify.py` 复算物理与费用，`src/v2_delivery.py` 生成结果工作簿，`src/v2_report.py` 生成正文。原价格预测由 `src/q4_forecast.py` 及其权重和日志复现，原日前保留方案由 `src/q12.py`、`src/q4_run.py` 复现。完整可运行源程序随支撑材料提供，不用截图替代代码。复现顺序见 `docs/RUN_V2.md`。

src/v2_model.py

```
"""Exact LP reformulation and causal forecast-corrected dispatch.

Energy in kWh. Same settlement, time axis and realtime regulation as V1.
No training or tuning is performed here; inputs explicitly carry decisions.
"""

from dataclasses import dataclass, asdict
from functools import lru_cache
import time
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from src.q12 import B, TOL, audit_trace, execute_day

@lru_cache(maxsize=32)
def structure(n, revision):
    # g,c,d,w,S[increase,decrease]; state has n+1 boundary values.
    size = (7 if revision else 5)*n+1
    a = lil_matrix(((3 if revision else 2)*n, size))
    for t in range(n):
        a[t,t], a[t,n+t], a[t,2*n+t], a[t,3*n+t] = 1,-1,1,-1
        a[n+t,4*n+t+1], a[n+t,4*n+t] = 1,-1
        a[n+t,n+t], a[n+t,2*n+t] = -B.eta_c,1/B.eta_d
        if revision:
            a[2*n+t,t], a[2*n+t,5*n+1+t], a[2*n+t,6*n+1+t] = 1,-1,1
    return a.tocsr()

def remove_cycles(charge, discharge, spill):
    """Preserve grid purchases and ALL SOC values, eliminate simultaneous flow.

    x=min(c,d/(eta_c*eta_d)); c'=c-x; d'=d-eta_c*eta_d*x;
    w'=w+(1-eta_c*eta_d)*x. Requires free unbounded curtailment.
    """
    rho=B.eta_c*B.eta_d
    x=np.minimum(charge,discharge/rho)
    return charge-x,discharge-rho*x,spill+(1-rho)*x

def solve(load,pv,price,initial,terminal=6000.,old=None,refund=True,equal=False):
    load,pv,price=map(lambda x:np.asarray(x,dtype=float),(load,pv,price))
    n=len(price);revision=old is not None;size=(7 if revision else 5)*n+1
    if not all(x.shape==(n,) and np.isfinite(x).all() for x in (load,pv,price)):
        raise ValueError('Nonfinite or misaligned dispatch inputs')
    if np.any(price<=0) or not B.minimum<=terminal<=B.maximum:
        raise ValueError('Positive prices and admissible terminal storage required')
    bounds=[(0,None)]*size
    bounds[n:3*n]=[(0,B.limit)]*(2*n)
    bounds[4*n:5*n+1]=[(B.minimum,B.maximum)]*(n+1)
    bounds[4*n]=(initial,initial);bounds[5*n]=(terminal,terminal if equal else B.maximum)
    cost=np.zeros(size)
    rhs=np.r_[load-pv,np.zeros(n)]
    if revision:
        old=np.asarray(old,dtype=float)
        assert old.shape==(n,) and np.isfinite(old).all() and old.min()>=-TOL
        cost[5*n+1:6*n+1]=1.5*price
        cost[6*n+1:]=(-.5 if refund else .5)*price
        bounds[6*n+1:]=(0,max(float(v),0)) for v in old
        rhs=np.r_[rhs,old]
    else:cost[:n]=price
    t=time.perf_counter()
    fit=linprog(cost,A_eq=structure(n,revision),b_eq=rhs,bounds=bounds,method='highs',
        options={'time_limit':30,'dual_feasibility_tolerance':1e-
8,'primal_feasibility_tolerance':1e-8})
    if not fit.success:raise RuntimeError(f'LP status={fit.status}: {fit.message}')
    x=fit.x;c,d,w=remove_cycles(x[n:2*n],x[2*n:3*n],x[3*n:4*n])
    f=pd.DataFrame(dict(plan_kwh=x[:n],charge_kwh=c,discharge_kwh=d,spill_kwh=w,
        soc_start_kwh=x[4*n:5*n],soc_end_kwh=x[4*n+1:5*n+1],load_kwh=load,pv_kwh=pv,
        emergency_kwh=np.zeros(n),price=price))
    physical=audit_trace(f,initial=initial)
    return f,dict(objective=float(fit.fun),seconds=time.perf_counter()-t,status=int(fit.status),
```

```

iterations=int(fit.nit),physical=physical)

@dataclass(frozen=True)
class Policy:
    quantile:float=.7
    terminal:float=6000.
    mask:int=0
    beta:float=0.
    refund:bool=True
    lookback:int=18
    decay:int=36

def corrected_load(data,pred,beta,lookback=18,decay=36):
    """At each release, use only the just-completed <=3h load residuals.

    No observed PV is used to correct load; no current/future interval enters.
    Correction decays with lead, preventing all-day propagation of a short shock.
    """
    out=np.repeat(pred['load'][:,None,:],4,axis=1).copy()
    for v in range(1,4):
        start=36*v;left=max(0,start-lookback)
        err=(data['load'][:,left:start]-pred['load'][:,left:start]).mean(axis=1)
        lead=np.arange(144-start)+.5
        out[:,v,start:]=np.maximum(0,out[:,v,start:]+beta*err[:,None]*np.exp(-lead[None,:]/decay))
    return out

def risk_buffers(data,load_pred,pv_pred,q):
    out=np.zeros_like(load_pred)
    if q==0:return out
    for day in range(7,len(load_pred)):
        left=max(1,day-28)
        error=data['load'][:,left:day,None,:]-data['pv'][:,left:day,None,:]-load_pred[left:day]+pv_pred[left:day]
        for v in range(4):
            start=36*v
            # Historical same-release residuals, not errors from unreleased versions.
            for slot in range(start,144):
                out[day,v,slot]=max(0,float(np.quantile(error[:,v,max(start,slot-3):min(144,slot+4)],q)))
    return out

def run(data,load_pred,pv_pred,buffers,policy,*,start,stop,initial,name,prices=None,retain=True):
    state=float(initial);frames=[];days=[];ledger=[];solves=[]
    for day in range(start,stop):
        date=str(data['dates'][day].date());s0=state
        realized=data['price'] if prices is None else data['actual_price'][day]
        p0=data['price'] if prices is None else prices[day,0,:144]
        sol,meta=solve(load_pred[day,0]+buffers[day,0],pv_pred[day,0],p0,state,policy.terminal)
        solves.append({'date':date,'version':0,**{k:meta[k] for k in
['seconds','status','iterations','objective']})})
        plan=np.maximum(sol.plan_kwh.to_numpy(),0);original=plan.copy()
        up=np.zeros(144);down=np.zeros(144);blocks=[]
        for v in range(4):
            first=v*36;last=first+36
            if v and policy.mask & (1<<(v-1)):
                p=data['price'][first:] if prices is None else prices[day,v,:144-first]
                old=plan[first:].copy()
                sol,meta=solve(load_pred[day,v,first:]+buffers[day,v,first:],pv_pred[day,v,first:],
                    p,state,policy.terminal,old,policy.refund)
                solves.append({'date':date,'version':v,**{k:meta[k] for k in
['seconds','status','iterations','objective']})})
                new=np.maximum(sol.plan_kwh.to_numpy(),0);delta=new-old
                up[first:]+=np.maximum(delta,0);down[first:]+=np.maximum(-delta,0);plan[first:]=new
                if retain:
                    for j,slot in enumerate(range(first,144)):
                        ledger.append([date,v,first,slot,float(old[j]),float(new[j]),max(float(delta[j]),0),
                            max(-float(delta[j]),0),float(p[j]),float(load_pred[day,v,slot]),
                                float(pv_pred[day,v,slot]),float(buffers[day,v,slot]),state)])
    f=execute_day(plan[first:last],data['load'][day,first:last],data['pv'][day,first:last],state,realized[first:
last])
    state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)

```

```

f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',np.arange(144))
f['original_plan_kwh']=original;f['increase_kwh']=up;f['decrease_kwh']=down
f['original_cost_yuan']=realized*original;f['increase_cost_yuan']=realized*1.5*up
f['decrease_cost_yuan']=realized*(-.5 if policy.refund else .5)*down
f['emergency_cost_yuan']=realized*5*f.emergency_kwh

f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
np.testing.assert_allclose(f.plan_kwh,original+up-down,atol=TOL,rtol=0)
item={'date':date,'strategy':name,'soc_start_kwh':s0,'soc_end_kwh':state}
for c in f:
    if c.endswith('_yuan') or c in ['original_plan_kwh','plan_kwh','increase_kwh','decrease_kwh',
        'emergency_kwh','spill_kwh','charge_kwh','discharge_kwh']:item[c]=float(f[c].sum())
    item['emergency_intervals']=int((f.emergency_kwh>TOL).sum());days.append(item)
    if retain:frames.append(f)
daily=pd.DataFrame(days);trace=pd.concat(frames,ignore_index=True) if retain else None
versions=pd.DataFrame(ledger,columns=['date','version','effective_slot','slot','old_kwh','new_kwh',
'increase_kwh','decrease_kwh','forecast_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision'
_soc_kwh'])
summary={'strategy':name,'policy':asdict(policy),'days':stop-
start,'initial_soc':initial,'final_soc':state,
'solver_seconds':sum(x['seconds'] for x in solves),'solver_calls':len(solves),
'physical':audit_trace(trace,initial=initial) if retain else None,
**{c:float(daily[c].sum()) for c in daily if c.endswith('_yuan') or c in
['emergency_kwh','spill_kwh']}}
return daily,trace,versions,summary,pd.DataFrame(solves)

```

src/v2_experiments.py

```

"""January-only selection, full-year paired ablations and frozen V2 policies."""
from pathlib import Path
from dataclasses import asdict
import argparse,hashlib,json,time,platform
import numpy as np
import pandas as pd
import scipy
from src.q12 import forecasts,dump_json,optimize_day
from src.q34_data import read_extended
from src.v2_model import Policy,corrected_load,risk_buffers,run,solve

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True)
    ap.add_argument('--out',type=Path,default=Path('artifacts/v2'));ap.add_argument('--
smoke',action='store_true')
    args=ap.parse_args();out=args.out;out.mkdir(parents=True,exist_ok=True);started=time.perf_counter()
    data,audit=read_extended(args.data_root);pred,fits=forecasts(data,'ridge',(1.,1.))
    price=np.load('artifacts/q34/price_forecasts.npz');sval=6244.256891388887;s0=9902.811287870369
    stop=38 if args.smoke else 365
    contract={'q2_quantiles':[0.,.5,.7,.8,.9],'q2_terminals':[1200.,6000.,9000.],
        'q3_quantiles':[0.,.5,.7],'q3_betas':[0.,.5,1.],'q3_masks':[6,7],
        'q3_terminal':6000.,'bias_lookback_slots':18,'bias_decay_slots':36,
        'price_models':['seasonal','ridge','gru_mean'],'validation':[21,31],'evaluation':[31,stop],
        'selection':'January realized total cost; ties use smaller complexity, no evaluation-based retuning',
        'scope':'Within-dataset revision; old full-year metrics were already inspected. Not new external
validation.'}
    dump_json(out/'design_before_evaluation.json',contract)
    loads={b:corrected_load(data,pred,b) for b in contract['q3_betas']}
    pv2=np.repeat(pred['pv'][:,None,:],4,axis=1);pv3=data['pv_issued'];cache={}
    def inputs(mode,policy,until):
        key=(mode,policy.beta,policy.quantile,until)
        pp=pv2 if mode==2 else pv3;ll=loads[policy.beta]
        if key not in cache:
            cache[key]=risk_buffers(data,ll[:until],pp[:until],policy.quantile)
        return ll,pp,cache[key]
    def calc(mode,pol,start,end,name,prices=None,retain=False):
        ll,pp,rr=inputs(mode,pol,end)
        return run(data,ll,pp,rr,pol,start=start,stop=end,initial=sval if start==21 else s0,
            name=name,prices=prices,retain=retain)
    selected={};validation=[]
    for mode in [2,3]:
        if mode==2:
            policies=[Policy(quantile=q,terminal=t) for q in contract['q2_quantiles'] for t in
contract['q2_terminals']]
        else:
            policies=[Policy(quantile=q,beta=b,mask=m) for q in contract['q3_quantiles']
                for b in contract['q3_betas'] for m in contract['q3_masks']]
        rows=[]
        for pol in policies:
            result=calc(mode,pol,21,31,f'q{mode}_validation')
        rows.append({**asdict(pol),'cost':result[3]['total_cost_yuan'],'final_soc':result[3]['final_soc'],
            'emergency_kwh':result[3]['emergency_kwh']})
        best=min(rows,key=lambda x:(x['cost'],x['beta'],x['mask'],x['quantile'],x['terminal']))
        selected[str(mode)]=Policy(**{k:best[k] for k in asdict(Policy())})
        validation.extend([{'mode':mode,**r} for r in rows])
        print('JANUARY SELECTION',mode,best,flush=True)
    pd.DataFrame(validation).to_csv(out/'validation.csv',index=False,float_format='%.17g')
    price_validation=[];price_selected={}
    for mode in [2,3]:
        for model in contract['price_models']:
            r=calc(mode,selected[str(mode)],21,31,f'q4_{mode}_{model}',price[model])
    price_validation.append({'mode':mode,'model':model,'cost':r[3]['total_cost_yuan'],'final_soc':r[3]['final_so
c']})
    price_selected[str(mode)]=min([r for r in price_validation if r['mode']==mode],key=lambda
x:x['cost'])['model']
    dump_json(out/'selection.json',{'policies':{k:asdict(v) for k,v in
selected.items()},'prices':price_selected,
        'frozen_before_evaluation':True,'validation_initial_soc':sval,'evaluation_initial_soc':s0})
    pd.DataFrame(price_validation).to_csv(out/'price_validation.csv',index=False,float_format='%.17g')
    q1,m=solve(data['q1_load'],data['q1_pv'],data['price'],6000,equal=True)
    q1.insert(0,'slot',range(144));q1.to_csv(out/'q1_intervals.csv',index=False,float_format='%.17g')
    _,oldm=optimize_day(data['q1_load'],data['q1_pv'],data['price'],6000,(6000,'equal'))

```

```

assert abs(m['objective']-oldm['cost'])<1e-5
dump_json(out/'q1_comparison.json',{'lp':m,'milp':oldm,'absolute_cost_difference':abs(m['objective']-oldm['cost'])})
summaries=[]
def save(mode,pol,label,prices=None):
    r=calc(mode,pol,31,stop,label,prices,True)
    daily,trace,versions,summary,solver=r
    daily.to_csv(out/f'{label}_daily.csv',index=False,float_format='%.17g')
    trace.to_csv(out/f'{label}_intervals.csv.gz',index=False,float_format='%.17g')
    if len(versions):versions.to_csv(out/f'{label}_versions.csv.gz',index=False,float_format='%.17g')
    solver.to_csv(out/f'{label}_solvers.csv.gz',index=False,float_format='%.17g')
    dump_json(out/f'{label}_summary.json',summary);summaries.append(summary)

print('EVALUATED',label,round(summary['total_cost_yuan'],2),'emergency',round(summary['emergency_kwh'],2),flush=True)
# LP-only controls distinguish solver equivalence from forecast/policy gains.
save(2,Policy(),'q2_lp_control')
save(3,Policy(quantile=0,mask=6),'q3_lp_control')
save(2,selected['2'],'q2_v2')
save(3,selected['3'],'q3_v2')
p=selected['3'];save(3,Policy(**{**asdict(p),'beta':1. if p.beta==0 else 0.}),'q3_bias_ablation')
save(3,Policy(**{**asdict(p),'refund':False}),'q3_no_refund')
for mode in [2,3]:
    for model in contract['price_models']:
        save(mode,selected[str(mode)],f'q4_{mode}_{model}',price[model])
dump_json(out/'summary.json',summaries)
# Same target clocks, separate release metrics; do not count repeated targets as independent.
metrics=[]
for model in ['seasonal','ridge','gru17','gru42','gru2026','gru_mean']:
    for version in range(4):
        actual=data['actual_price'][31:stop,version*36:]
        err=price[model][31:stop,version,:actual.shape[1]]-actual
        metrics.append({'model':model,'issue_hour':6*version,'n_pairs':err.size,
            'mae':float(np.abs(err).mean()),'rmse':float(np.sqrt((err*err).mean()))})
pd.DataFrame(metrics).to_csv(out/'price_errors_by_issue.csv',index=False,float_format='%.17g')
dump_json(out/'run_manifest.json',{'stage':'smoke' if args.smoke else
'full','seconds':time.perf_counter()-started,

'python':platform.python_version(),'numpy':np.__version__,'scipy':scipy.__version__,'input_files':audit['input_files'],
'source_sha256':{s:hashlib.sha256(Path(s).read_bytes()).hexdigest() for s in
['src/v2_model.py','src/v2_experiments.py']},

'forecast_archive_sha256':hashlib.sha256(Path('artifacts/q34/price_forecasts.npz').read_bytes()).hexdigest(),
'load_pv_forecast_fits':fits,'design':contract})
print('COMPLETE',round(time.perf_counter()-started,2),flush=True)

if __name__=='__main__':main()

```


src/v2_verify.py

```

"""Independent saved-output audit; raw XLSX equality, accounting and paired effects."""
from pathlib import Path
import argparse,hashlib,json
import numpy as np
import pandas as pd
from src.q12 import B,TOL,dump_json,audit_trace
from src.q34_data import read_extended

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True)
    ap.add_argument('--out',type=Path,default=Path('artifacts/v2'));args=ap.parse_args();out=args.out
    data,raw=read_extended(args.data_root);checks={}
    manifest=json.loads((out/'run_manifest.json').read_text(encoding='utf8'))
    assert manifest['stage']=='full'
    for file,h in manifest['source_sha256'].items():assert
hashlib.sha256(Path(file).read_bytes()).hexdigest()==h
    for path in sorted(out.glob('*_intervals.csv.gz')):
        name=path.name.removesuffix('_intervals.csv.gz');f=pd.read_csv(path)
        s=json.loads((out/f'{name}_summary.json').read_text());d=pd.read_csv(out/f'{name}_daily.csv')
        assert len(f)==334*144 and f.date.nunique()==334
        np.testing.assert_array_equal(f.date,np.repeat(data['dates'][31:].strftime('%Y-%m-%d'),144))
        np.testing.assert_array_equal(f.slot,np.tile(np.arange(144),334))
        for k in ['load','pv']:np.testing.assert_allclose(f[k+'_kwh'],data[k][31:].ravel(),rtol=0,atol=1e-8)
        prices=data['actual_price'][31:].ravel() if name.startswith('q4') else np.tile(data['price'],334)
        np.testing.assert_allclose(f.price,prices,rtol=0,atol=1e-12)
        physical=audit_trace(f,initial=s['initial_soc'])
        vp=out/f'{name}_versions.csv.gz';v=pd.read_csv(vp) if vp.exists() else pd.DataFrame()
        groups={date:g for date,g in v.groupby('date')} if len(v) else {}
        for date,g in f.groupby('date',sort=False):
            plan=g.original_plan_kwh.to_numpy().copy();up=np.zeros(144);down=np.zeros(144)
            if date in groups:
                for ver,items in groups[date].groupby('version',sort=True):
                    first=int(ver)*36;slots=items.slot.to_numpy(int)
                    np.testing.assert_array_equal(slots,np.arange(first,144))
                    assert (items.effective_slot==first).all()
                    np.testing.assert_allclose(items.old_kwh,plan[slots],atol=TOL,rtol=0)
        np.testing.assert_allclose(items.decision_soc_kwh,g.soc_start_kwh.iloc[first],atol=TOL,rtol=0)
        delta=items.new_kwh.to_numpy()-plan[slots]
        np.testing.assert_allclose(items.increase_kwh,np.maximum(delta,0),atol=TOL,rtol=0)
        np.testing.assert_allclose(items.decrease_kwh,np.maximum(-delta,0),atol=TOL,rtol=0)
        up[slots]+=np.maximum(delta,0);down[slots]+=np.maximum(-
delta,0);plan[slots]=items.new_kwh
        for a,b in [(plan,g.plan_kwh),(up,g.increase_kwh),(down,g.decrease_kwh)]:
            np.testing.assert_allclose(a,b,atol=TOL,rtol=0)
            coeff=-.5 if s['policy']['refund'] else .5
    pieces={'original_cost_yuan':prices*f.original_plan_kwh,'increase_cost_yuan':prices*1.5*f.increase_kwh,
'decrease_cost_yuan':prices*coeff*f.decrease_kwh,'emergency_cost_yuan':prices*5*f.emergency_kwh}
    pieces['total_cost_yuan']=sum(pieces.values())
    for key,values in pieces.items():
        np.testing.assert_allclose(f[key],values,rtol=0,atol=TOL)
        np.testing.assert_allclose(d[key],f.groupby('date',sort=False)[key].sum(),rtol=0,atol=TOL)
        assert abs(float(values.sum())-s[key])<1e-5
    ss=pd.read_csv(out/f'{name}_solvers.csv.gz');assert (ss.status==0).all()

    checks[name]={'pass':True,'physical':physical,'solves':len(ss),'revision_rows':len(v),'cost':s['total_cost_yuan']}
    assert len(checks)==12
    q1=pd.read_csv(out/'q1_intervals.csv');audit_trace(q1,initial=6000)
    assert abs(q1.soc_end_kwh.iloc[-1]-6000)<TOL
    assert abs(np.dot(q1.plan_kwh,data['price'])-
json.loads(Path('artifacts/q12/q1_summary.json').read_text())['cost'])<TOL
    # Pair days, retain autocorrelation within circular blocks; empirical uncertainty only.

    sel=json.loads((out/'selection.json').read_text());comparison=[];monthly=[];rng=np.random.default_rng(202609
11)
    mappings={'q2':('q2_v2','artifacts/q12/ridge_q0.7_intervals.csv.gz'),
'q3':('q3_v2','artifacts/q34/q3_m6_intervals.csv.gz'),
'q4-2':(f'q4_2_{sel["prices"]['2"]}','artifacts/q34/q4_2_ridge_intervals.csv.gz'),
'q4-3':(f'q4_3_{sel["prices"]['3"]}','artifacts/q34/q4_3_seasonal_intervals.csv.gz')}
    for key,(new,oldpath) in mappings.items():
        before=pd.read_csv(oldpath);after=pd.read_csv(out/f'{new}_intervals.csv.gz')

```

```

        if 'total_cost_yuan' not in
before: before['total_cost_yuan']=before.price*(before.plan_kwh+5*before.emergency_kwh)

c0=before.groupby('date',sort=False).total_cost_yuan.sum();c1=after.groupby('date',sort=False).total_cost_yuan.sum()
assert c0.index.equals(c1.index)
diff=(c0-c1).to_numpy();ci={}
for block in [7,14,28]:
    starts=rng.integers(0,len(diff),(2000,int(np.ceil(len(diff)/block))))
    idx=((starts[:,None]+np.arange(block))%len(diff)).reshape(2000,-1)[:,:len(diff)]
    totals=diff[idx].mean(axis=1)*len(diff)
    ci[str(block)]=[float(x) for x in np.quantile(totals,[.025,.975])]
    endgap=float(before.soc_end_kwh.iloc[-1]-after.soc_end_kwh.iloc[-1])
    # Accounting stress range only: no actual post-year trades are fabricated.
    replacement=max(endgap,0)/B.eta_c*float(data['actual_price'].max() if key.startswith('q4') else
data['price'].max())

comparison.append({'question':key,'candidate':new,'v1_cost':float(c0.sum()),'candidate_cost':float(c1.sum()),
'saving_yuan':float(diff.sum()),'saving_percent':float(diff.sum()/c0.sum()*100),
'v1_final_soc':float(before.soc_end_kwh.iloc[-1]),'candidate_final_soc':float(after.soc_end_kwh.iloc[-1]),
'terminal_inventory_stress_yuan':replacement,'saving_after_inventory_stress':float(diff.sum()-replacement),
'block_bootstrap_95pct':ci,'winning_days':int((diff>0).sum()),'losing_days':int((diff<0).sum())})
for month,g in
pd.DataFrame({'date':c0.index,'v1':c0.to_numpy(),'candidate':c1.to_numpy()}).groupby(lambda
i:c0.index[i][7]):

monthly.append({'question':key,'month':month,'v1':float(g.v1.sum()),'candidate':float(g.candidate.sum()),'saving':float((g.v1-g.candidate).sum())})

dump_json(out/'paired_comparison.json',comparison);pd.DataFrame(monthly).to_csv(out/'monthly_comparison.csv',index=False,float_format='%0.17g')

dump_json(out/'verification.json',{'pass':True,'traces':checks,'q1_pass':True,'input_files':raw['input_files'],
'uncertainty_note':'Circular paired-day bootstrap; assumes block exchangeability, not an external-year confidence guarantee; length sensitivity 7/14/28 days.',
'verifier_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()})
print(json.dumps(comparison,ensure_ascii=False,indent=2));print('PASS: 12 full-year traces + Q1; all saved costs, revisions, physical constraints and source hashes')

if __name__=='__main__':main()

```

src/v2_delivery.py

```
"""Template-preserving V2 workbooks and complete requested-date tables."""
from pathlib import Path
import argparse,hashlib,json,shutil
import re
from copy import copy
from datetime import datetime
import numpy as np
import pandas as pd
import openpyxl
from openpyxl.styles import Font,PatternFill,Alignment
from src.q12 import interval_label,dump_json
from src.deliver_q12 import blocks,emergency_runs

OUT=Path('artifacts/v2')
SOURCES={'1':'artifacts/q12/q1_intervals.csv','2':'artifacts/q12/ridge_q0.7_intervals.csv.gz',
        '3':'artifacts/v2/q3_v2_intervals.csv.gz','4-2':'artifacts/q34/q4_2_ridge_intervals.csv.gz',
        '4-3':'artifacts/v2/q4_3_seasonal_intervals.csv.gz'}

def load_trace(key):
    f=pd.read_csv(SOURCES[key])
    if 'original_plan_kwh' not in f:f['original_plan_kwh']=f.plan_kwh
    if 'total_cost_yuan' not in f:f['total_cost_yuan']=f.price*(f.plan_kwh+5*f.emergency_kwh)
    return f

def table(title,headers,rows):
    def fmt(x):return f'{x:,.2f}' if isinstance(x,(float,np.floating)) else str(x)
    return '对应结果见'+title.split(' ',2)[0]+' '+title.split(' ',2)[1]+'。 \n\n'+title+'\n\n| '+' |
    '.join(headers)+' |\n| '+' | '.join(['---']*len(headers))+ ' |\n'+'.join('| '+' | '.join(map(fmt,r))+' |\n'
    for r in rows)+'\n'

def specified_tables():
    text=['## 附录 A 四问指定日期完整结果\n\n电量单位为 kWh，费用单位为元。普通购电与紧急购电分列。滚动问题的指定时
    段和全天购电量指最终有效普通计划；午夜计划及每次更新完整保存在 Excel 中。所有表由实际轨迹自动生成。 \n']
    manifest=[]
    for key in SOURCES:
        f=load_trace(key);dates=['单日'] if key=='1' else ['2025-03-20','2025-06-21','2025-09-23','2025-12-
    21']
        text.append(f'### A-{{key}} 问题{{key}}\n')
        for date in dates:
            g=f if key=='1' else f[f.date==date]
            rows=[]
            for slots in [[60,72,84],[96,108,120]]:
                row=[]
                for slot in slots:row.extend([interval_label(slot),float(g.plan_kwh.iloc[slot])])
                rows.append(row)
            rows.append(['全天购电量',float(g.plan_kwh.sum()),'全天购电费',float(g.total_cost_yuan.sum()),'-
    ', '-'])
            text.append(table(f'表 A-{{key}}-{{date}}-1 指定时段购电及全天费用',[ '时间段','购电量']*3,rows))
            b=blocks(g);rows=[[*b[i],*b[i+1]] for i in range(0,6,2)]
            rows.append(['00:00 储电',float(g.soc_start_kwh.iloc[0]),'-','24:00 储电
    ',float(g.soc_end_kwh.iloc[-1]),'-'])
            text.append(table(f'表 A-{{key}}-{{date}}-2 储能运行',[ '时间段','充电量','放电量']*2,rows))
            if key!='1':text.append(table(f'表 A-{{key}}-{{date}}-3 紧急购电',[ '时段','电量'],emergency_runs(g) or
    [['无',0.])))
            manifest.append({'question':key,'date':date,'slots':[60,72,84,96,108,120],'blocks':6,
                'emergency_events':len(emergency_runs(g)),'cost':float(g.total_cost_yuan.sum())})
            dump_json(OUT/'specified_tables_manifest.json',manifest)
            combined='\n'.join(text)
            labels=re.findall(r'^表 (A-S+)',combined,re.M)
            for i,label in enumerate(labels,16):combined=combined.replace('表 '+label,'表 '+str(i))
            return combined

def export(root):
    # Accepted revisions are a within-dataset engineering choice. Preserve both rejected candidates.
```

```

        accepted={'1':'V1 retained (same optimal cost)','2':'V1 retained: new terminal candidate deteriorated',
        '3':'January-selected q=.5, terminal=6000, mask=6, beta=0',
        '4-2':'V1 retained: new terminal candidate deteriorated','4-3':'January-selected rolling q=.5 with
seasonal prices'}
        dump_json(OUT/'delivery_selection.json',{'sources':SOURCES,'decisions':accepted,
        'interpretation':'V2 release retains V1 where revision failed; release acceptance is not a new
untouched test-set selection.'})
        audits={}
        for key in SOURCES:
            if key in ['1','2','4-2']:
                src=Path('artifacts/q12' if key in ['1','2'] else 'artifacts/q34')/f'result{key}.xlsx'

shutil.copy2(src,OUT/src.name);audits[key]={'copied_verified_v1':True,'sha256':hashlib.sha256(src.read_bytes(
)).hexdigest()}
            continue
            wb=openpyxl.load_workbook(root/f'附件/附件 5/result{key}.xlsx')
            f=load_trace(key);payload={};plans=[];adjusted=[];storage=[];emergency=[];costs=[]
            for date,g in f.groupby('date',sort=False):
                dt=datetime.strptime(date,'%Y-%m-%d')

plans.append([dt,*g.original_plan_kwh.tolist(),float(g.original_plan_kwh.sum()),float(g.original_cost_yuan.s
um())])
                adjusted.append([dt,*g.plan_kwh.tolist(),float(g.plan_kwh.sum()),float(g.total_cost_yuan.sum())])
                for i,b in enumerate(blocks(g)):
                    storage.append([dt if i==0 else None,*b,'00:00' if i==0 else '24:00' if i==1 else None,
                    float(g.soc_start_kwh.iloc[0]) if i==0 else float(g.soc_end_kwh.iloc[-1]) if i==1 else
None])
                for i,e in enumerate(emergency_runs(g) or [['无',0]]):emergency.append([dt if i==0 else
None,*e])
                costs.append([dt,*[float(g[c].sum()) for c in
['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','total_cost_yuan']]])
                head=['日期',*[interval_label(i) for i in range(144)],'全天购电量','全天购电费']
                payload.update({'计划购电量':[head,*plans],'调整购电量':[head,*adjusted],
                '充放电量':[['日期','时间段','充电量','放电量','时刻','储电量'],*storage],
                '紧急购电量':[['日期','时间段','电量'],*emergency],
                '费用分解':[['日期','午夜计划费','增购费','减购净费用','紧急费','总费'],*costs]})
                vp=Path(SOURCES[key].replace('_intervals','_versions'));v=pd.read_csv(vp);vr=[]
                for (date,ver),g in v.groupby(['date','version'],sort=True):
                    values=[None]*144
                    for r in g.itertuples():values[int(r.slot)]=float(r.new_kwh)
                    vr.append([datetime.strptime(date,'%Y-%m-%d'),f'{int(ver)*6*24:02d}:00',*values])
                payload['计划版本']=[['日期','发布时间'],*head[1:145]],*vr]
                payload['口径说明']=[['事项','约定'],['版本',accepted[key]],['时间','按输入右端点解释为十分钟区间，输出标
签 00:00-24:00'],
                ['费用','午夜计划+逐次增购 1.5 倍-减购 0.5 倍+紧急 5 倍；按交付区间实际价'],
                ['调整购电量','最终有效普通计划，非调整差值；已执行区间在版本表留空'],['储能','母线侧电量，两侧效率各 0.9，
实际 SOC 跨日连续']]
                for title,rows in payload.items():
                    ws=wb[title] if title in wb.sheetnames else wb.create_sheet(title)
                    for row in ws:
                        for cell in row:cell.value=None
                    for i,row in enumerate(rows,1):
                        for j,val in enumerate(row,1):
                            c=ws.cell(i,j,val)
                            if i==1:c.font=Font(name='Microsoft
YaHei',bold=True,color='FFFFFF');c.fill=PatternFill('solid',fgColor='3E608D')
                            elif isinstance(val,datetime):c.number_format='yyyy-mm-dd'
                            elif isinstance(val,(int,float)):c.number_format='0.00'
                    ws.freeze_panes='B2';ws.column_dimensions['A'].width=16;ws.column_dimensions['B'].width=22
                    dest=OUT/f'result{key}.xlsx';wb.save(dest);wb.close()
                    reread=openpyxl.load_workbook(dest,data_only=False);count=0
                    for sheet,rows in payload.items():
                        for i,row in enumerate(rows,1):
                            for j,expected in enumerate(row,1):
                                actual=reread[sheet].cell(i,j).value
                                if isinstance(expected,(int,float)):
                                    assert isinstance(actual,(int,float)) and np.isclose(actual,expected,atol=1e-
7,rtol=1e-12)
                                else:assert actual==expected,(sheet,i,j,actual,expected)
                                count+=1
                    assert not any(c.data_type=='e' for ws in reread for row in ws for c in row)

reread.close();audits[key]={'cells_compared':count,'pass':True,'sha256':hashlib.sha256(dest.read_bytes()).he
xdigest()}

```

```
    dump_json(OUT/'xlsx_audit.json',audits);print('Five result workbooks verified',
{k:v.get('cells_compared','V1 exact copy') for k,v in audits.items()})

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--data-
root',type=Path,required=True);a=p.parse_args();export(a.data_root)
```

src/q12.py

"""Causal forecasts, MILP day-ahead dispatch and independent physical audits.

Run from the repository: python -m src.q12 --data-root <C 题 directory> --stage full
All power observations are interpreted as interval means at right-end labels.
"""

```
from __future__ import annotations
```

```
import argparse
from dataclasses import asdict, dataclass
import hashlib
import json
import os
from pathlib import Path
import platform
import subprocess
import time
```

```
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
import numpy as np
import pandas as pd
import scipy
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import lil_matrix
```

```
@dataclass(frozen=True)
```

```
class Battery:
```

```
    minimum: float = 1200.0
    maximum: float = 10800.0
    initial: float = 6000.0
    max_power: float = 5000.0
    eta_c: float = 0.9
    eta_d: float = 0.9
    dt: float = 1 / 6
```

```
    @property
```

```
    def limit(self):
        return self.max_power * self.dt
```

```
B = Battery()
```

```
N = 144
```

```
TOL = 1e-5
```

```
def dump_json(path, obj):
```

```
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")
```

```
def interval_label(i):
```

```
    def clock(m):
        return f"{m // 60:02d}:{m % 60:02d}"
    return f"{clock(i * 10)}-{clock((i + 1) * 10)}"
```

```
def time_minutes(value):
```

```
    if hasattr(value, "hour"):
        return value.hour*60 + value.minute
    text = str(value).strip()
    next_day = "+1" in text
    hour, minute, *_ = text.replace("+1", "").split(":")
    return int(hour)*60 + int(minute) + 1440*int(next_day)
```

```
def read_inputs(root):
```

```

root = Path(root)
day = pd.read_excel(root / "附件/附件 1.xlsx", sheet_name=0, header=0)
load = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="小区负载", header=0)
pv = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="光伏发电实际功率", header=0)
dates = pd.to_datetime(load.iloc[:, 0])
if day.shape != (144, 4) or load.shape != (365, 145) or pv.shape != (365, 145):
    raise ValueError("Unexpected input dimensions")
if not dates.equals(pd.to_datetime(pv.iloc[:, 0])):
    raise ValueError("Load/PV dates differ")
if list(dates) != list(pd.date_range("2025-01-01", "2025-12-31")):
    raise ValueError("Dates must cover the complete ordered year")
expected_minutes = list(range(10, 1441, 10))
if any([time_minutes(v) for v in labels] != expected_minutes
        for labels in (load.columns[1:], pv.columns[1:], day.iloc[:, 0])):
    raise ValueError("Input time labels do not match")
data = {
    "dates": pd.DatetimeIndex(dates), "price": day.iloc[:, 1].to_numpy(float),
    "q1_load": day.iloc[:, 2].to_numpy(float) * B.dt,
    "q1_pv": day.iloc[:, 3].to_numpy(float) * B.dt,
    "load": load.iloc[:, 1:].to_numpy(float) * B.dt,
    "pv": pv.iloc[:, 1:].to_numpy(float) * B.dt,
}
audit = {"shapes": {"attachment1": list(day.shape), "load": list(load.shape), "pv": list(pv.shape)},
        "date_start": str(dates.iloc[0].date()), "date_end": str(dates.iloc[-1].date()),
        "duplicates": int(dates.duplicated().sum()), "input_files": [], "variables": {}}
for key in ["price", "q1_load", "q1_pv", "load", "pv"]:
    a = data[key]
    if not np.isfinite(a).all() or (a < 0).any():
        raise ValueError(f"Nonfinite or negative data in {key}; investigate, do not silently impute")
    audit["variables"][key] = {"count": int(a.size), "missing": 0, "min": float(a.min()),
                              "max": float(a.max()), "zero_count": int((a == 0).sum())}
if (data["price"] <= 0).any():
    raise ValueError("This implementation assumes strictly positive fixed prices")
for relative in ["C 题.pdf", "附件/附件 1.xlsx", "附件/附件 2.xlsx"]:
    raw = (root / relative).read_bytes()
    audit["input_files"].append({"path": relative, "bytes": len(raw), "sha256":
hashlib.sha256(raw).hexdigest()})
audit["data_treatment"] = "No missing/negative values found; no observations deleted or imputed. Unusual
peaks retained."
return data, audit

```

```

def optimize_day(load, pv, price, initial, terminal=6000.0, battery=B, integer=True):
    """Nominal minimum-cost schedule, g/c/d/w/S/z in kWh; terminal is a lower bound.

```

```

    For Q1 initial=terminal and terminal_equal=True is obtained by the explicit
    upper bound below when requested by a (value, 'equal') tuple.
    """

```

```

    n = len(price)
    eq_terminal = isinstance(terminal, tuple)
    target = terminal[0] if eq_terminal else terminal
    size = 6 * n + 1
    gi, ci, di, wi, si, zi = 0, n, 2*n, 3*n, 4*n, 5*n+1
    objective = np.zeros(size)
    objective[:n] = price
    lb, ub = np.zeros(size), np.full(size, np.inf)
    ub[ci:wi] = battery.limit
    lb[si:zi], ub[si:zi] = battery.minimum, battery.maximum
    lb[si] = ub[si] = initial
    lb[si+n] = target
    if eq_terminal:
        ub[si+n] = target
    ub[zi:] = 1
    integrality = np.zeros(size)
    integrality[zi:] = int(integer)
    a = lil_matrix((4*n, size))
    lower, upper = np.zeros(4*n), np.zeros(4*n)
    for t in range(n):
        a[t, gi+t], a[t, ci+t], a[t, di+t], a[t, wi+t] = 1, -1, 1, -1
        lower[t] = upper[t] = load[t] - pv[t]
        a[n+t, si+t+1], a[n+t, si+t] = 1, -1
        a[n+t, ci+t], a[n+t, di+t] = -battery.eta_c, 1 / battery.eta_d
        a[2*n+t, ci+t], a[2*n+t, zi+t] = 1, -battery.limit
        lower[2*n+t], upper[2*n+t] = -np.inf, 0
        a[3*n+t, di+t], a[3*n+t, zi+t] = 1, battery.limit
        lower[3*n+t], upper[3*n+t] = -np.inf, battery.limit
    start = time.perf_counter()

```

```

result = milp(objective, integrality=integrality, bounds=Bounds(lb, ub),
              constraints=LinearConstraint(a.tocsc(), lower, upper),
              options={"time_limit": 30.0, "mip_rel_gap": 1e-7})
elapsed = time.perf_counter() - start
if not result.success or result.x is None:
    raise RuntimeError(f"MILP failed: status={result.status}, {result.message}")
x = result.x
sol = {"plan_kwh": x[:n], "charge_kwh": x[ci:di], "discharge_kwh": x[di:wi],
      "spill_kwh": x[wi:si], "soc_start_kwh": x[si:si+n], "soc_end_kwh": x[si+1:zi],
      "emergency_kwh": np.zeros(n), "load_kwh": np.asarray(load), "pv_kwh": np.asarray(pv)}
frame = pd.DataFrame(sol)
frame["price"] = price
meta = {"status": int(result.status), "cost": float(np.dot(price, x[:n])),
      "seconds": elapsed, "mip_gap": float(getattr(result, "mip_gap", 0) or 0),
      "dual_bound": float(getattr(result, "mip_dual_bound", result.fun) or result.fun)}
if integer:
    audit_trace(frame, battery=battery, initial=initial)
return frame, meta

def execute_day(plan, load, pv, initial, price, battery=B):
    """Causal real-time balancing; does not read future observations.

    Assumes instantaneous regulation within a ten-minute constant-power interval.
    Surplus charges first; deficit exhausts feasible discharge before emergency.
    """
    s = float(initial)
    records = []
    for t, (g, l, v) in enumerate(zip(plan, load, pv)):
        surplus = float(g + v - l)
        c = min(max(surplus, 0), battery.limit, max(0, (battery.maximum-s)/battery.eta_c))
        d = min(max(-surplus, 0), battery.limit, max(0, (s-battery.minimum)*battery.eta_d))
        e = max(0, -surplus-d)
        w = max(0, surplus-c)
        end = s + battery.eta_c*c - d/battery.eta_d
        records.append((g, l, v, c, d, e, w, s, end, price[t]))
        s = end
    frame = pd.DataFrame(records, columns=["plan_kwh", "load_kwh", "pv_kwh", "charge_kwh", "discharge_kwh",
      "emergency_kwh", "spill_kwh", "soc_start_kwh", "soc_end_kwh",
"price"], dtype=float)
    audit_trace(frame, battery=battery, initial=initial)
    return frame

def audit_trace(frame, battery=B, initial=None):
    """Independent vectorized equations, also applicable to saved full-year traces."""
    f = frame
    balance = f.plan_kwh + f.pv_kwh + f.discharge_kwh + f.emergency_kwh - f.load_kwh - f.charge_kwh -
f.spill_kwh
    evolution = f.soc_end_kwh - f.soc_start_kwh - battery.eta_c*f.charge_kwh + f.discharge_kwh/battery.eta_d
    continuity = f.soc_start_kwh.to_numpy()[1:] - f.soc_end_kwh.to_numpy()[:-1]
    metrics = {"max_balance_residual_kwh": float(np.max(np.abs(balance))),
      "max_soc_residual_kwh": float(np.max(np.abs(evolution))),
      "max_soc_discontinuity_kwh": float(np.max(np.abs(continuity))) if len(f)>1 else 0,
      "simultaneous_intervals": int(((f.charge_kwh > TOL) & (f.discharge_kwh > TOL)).sum()),
      "min_soc_kwh": float(min(f.soc_start_kwh.min(), f.soc_end_kwh.min())),
      "max_soc_kwh": float(max(f.soc_start_kwh.max(), f.soc_end_kwh.max()))}
    checks = [np.isfinite(f.select_dtypes(include="number").to_numpy()).all(),
      (f[["plan_kwh", "charge_kwh", "discharge_kwh", "emergency_kwh", "spill_kwh"]].to_numpy() >= -
TOL).all(),
      metrics["max_balance_residual_kwh"] < TOL, metrics["max_soc_residual_kwh"] < TOL,
      metrics["max_soc_discontinuity_kwh"] < TOL, metrics["simultaneous_intervals"] == 0,
      metrics["min_soc_kwh"] >= battery.minimum-TOL, metrics["max_soc_kwh"] <= battery.maximum+TOL,
      f.charge_kwh.max() <= battery.limit+TOL, f.discharge_kwh.max() <= battery.limit+TOL]
    if initial is not None:
        checks.append(abs(f.soc_start_kwh.iloc[0]-initial) < TOL)
    if not all(checks):
        raise AssertionError(f"Physical audit failed: {metrics}")
    return {"**metrics", "pass": True}

def features(history, day, dates):
    """Features for all 144 targets on a day, using completed days only."""
    if day < 7:
        raise ValueError("Seven completed days required")

```



```

past = history[day-7:day]
h = (np.arange(N)+0.5)/N
dow = dates[day].dayofweek
cols = [past[-1], past[-2], past[0], past[-3:].mean(0), past.mean(0), past.std(0),
        np.full(N, past[-1].mean()), np.full(N, past[-3:].mean())]
for k in (1, 2, 3):
    cols.extend([np.sin(2*np.pi*k*h), np.cos(2*np.pi*k*h)])
cols.extend([np.full(N, float(dow == k)) for k in range(7)])
return np.column_stack(cols)

def ridge_fit(x, y, alpha):
    """Centered ridge via linear solve; intercept unpenalized, no sklearn required."""
    mean, scale = x.mean(0), x.std(0)
    scale[scale < 1e-10] = 1
    z = (x-mean)/scale
    intercept = float(y.mean())
    coef = np.linalg.solve(z.T@z + alpha*np.eye(z.shape[1]), z.T@(y-intercept))
    return mean, scale, coef, intercept

def forecasts(data, method, alphas=(10.0, 10.0), stop=365):
    pred, fit_dates = {}, []
    for k, alpha in zip(("load", "pv"), alphas):
        values = data[k]
        out = np.zeros((stop, N))
        model, fitted_on = None, -1
        all_x = {d: features(values, d, data["dates"]) for d in range(7, stop)}
        for d in range(stop):
            if d == 0:
                out[d] = data[f"q1_{k}"]
            elif d < 7:
                out[d] = values[max(0,d-3):d].mean(0)
            elif method == "seasonal" or d < 14:
                out[d] = 0.5*values[d-1] + 0.5*values[d-7]
            else:
                if model is None or d-fitted_on >= 7:
                    training_days = range(max(7,d-60), d)
                    x = np.concatenate([all_x[j] for j in training_days])
                    y = values[max(7,d-60):d].ravel()
                    model = ridge_fit(x, y, alpha)
                    fitted_on = d
                    fit_dates.append({"variable": k, "decision_date": str(data["dates"][d].date()),
                                    "fit_until": str(data["dates"][d-1].date()), "alpha": alpha,
                                    "training_rows": len(y)})
                mean, scale, coef, intercept = model
                out[d] = ((all_x[d]-mean)/scale)@coef+intercept
            out[d] = np.maximum(out[d], 0)
            if k == "pv" and d:
                out[d, values[max(0,d-7):d].max(0) == 0] = 0
        pred[k] = out
    return pred, fit_dates

def reserve(data, forecast, day, quantile):
    if quantile == 0 or day < 7:
        return np.zeros(N)
    start = max(1, day-28)
    errors = (data["load"][start:day]-data["pv"][start:day]
              -forecast["load"][start:day]+forecast["pv"][start:day])
    # Pool +/- three slots; no wrap at the day boundary, no future residuals.
    return np.array([max(0.0, float(np.quantile(errors[:,max(0,t-3):min(N,t+4)], quantile))) for t in
                     range(N)])

def summarize_day(f, date, name, seconds, gap=0.0):
    normal = float(np.dot(f.plan_kwh, f.price))
    emergency = float(5*np.dot(f.emergency_kwh, f.price))
    return {"date": str(date.date()), "strategy": name, "normal_cost_yuan": normal,
            "emergency_cost_yuan": emergency, "total_cost_yuan": normal+emergency,
            "plan_kwh": float(f.plan_kwh.sum()), "emergency_kwh": float(f.emergency_kwh.sum()),
            "emergency_intervals": int((f.emergency_kwh>TOL).sum()), "spill_kwh": float(f.spill_kwh.sum()),
            "charge_kwh": float(f.charge_kwh.sum()), "discharge_kwh": float(f.discharge_kwh.sum()),
            "soc_start_kwh": float(f.soc_start_kwh.iloc[0]), "soc_end_kwh": float(f.soc_end_kwh.iloc[-1]),
            "solve_seconds": seconds, "mip_gap": gap}

```

```

def backtest(data, forecast, start, stop, initial, quantile, name, retain=True):
    s, daily, frames = initial, [], []
    for d in range(start, stop):
        buffer = reserve(data, forecast, d, quantile)
        nominal, meta = optimize_day(forecast["load"][d]+buffer, forecast["pv"][d], data["price"], s)
        f = execute_day(nominal.plan_kwh.to_numpy(), data["load"][d], data["pv"][d], s, data["price"])
        daily.append(summarize_day(f, data["dates"][d], name, meta["seconds"], meta["mip_gap"]))
        if retain:
            f.insert(0, "interval_start", pd.date_range(data["dates"][d], periods=N, freq="10min"))
            f.insert(1, "date", str(data["dates"][d].date()))
            f.insert(2, "slot", np.arange(N))
            f["forecast_load_kwh"], f["forecast_pv_kwh"], f["reserve_kwh"] = forecast["load"][d],
forecast["pv"][d], buffer
            frames.append(f)
            s = float(f.soc_end_kwh.iloc[-1])
            if retain and (d == start or data["dates"][d].day == 1):
                print(f"{name}: {data['dates'][d].date()}, cumulative cost={sum(x['total_cost_yuan'] for x in
daily):.2f}", flush=True)
            combined = pd.concat(frames, ignore_index=True) if frames else None
            if combined is not None:
                audit_trace(combined, initial=initial)
            return pd.DataFrame(daily), combined, s

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-root", type=Path, required=True)
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--stage", choices=["q1", "smoke", "full"], default="full")
    args = parser.parse_args()
    args.out.mkdir(parents=True, exist_ok=True)
    start = time.perf_counter()
    data, input_audit = read_inputs(args.data_root)
    dump_json(args.out/"input_audit.json", input_audit)
    q1, q1_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"))
    lp, lp_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial, "equal"),
integer=False)
    q1.insert(0, "slot", np.arange(N)); q1.insert(1, "interval", [interval_label(i) for i in range(N)])
    q1.to_csv(args.out/"q1_intervals.csv", index=False)
    no_storage_cost = float(np.dot(np.maximum(data["q1_load"]-data["q1_pv"], 0), data["price"]))
    q1_meta.update({"no_storage_cost": no_storage_cost, "saving_percent": 100*(no_storage_cost-
q1_meta["cost"])/no_storage_cost,
                    "lp_cost_lower_bound": lp_meta["cost"], "lp_gap_yuan": q1_meta["cost"]-lp_meta["cost"],
                    "audit": audit_trace(q1, initial=B.initial), "plan_kwh": float(q1.plan_kwh.sum())})
    dump_json(args.out/"q1_summary.json", q1_meta)
    print("Q1", json.dumps(q1_meta, ensure_ascii=False), flush=True)
    if args.stage == "q1":
        return
    seasonal, _ = forecasts(data, "seasonal")
    tuning = []
    for alpha in (1.0, 10.0, 100.0):
        pred, _ = forecasts(data, "ridge", (alpha, alpha), stop=31)
        row = {"alpha": alpha}
        for key in ("load", "pv"):
            row[key+"_mae_kw"] = float(np.abs(pred[key][21:31]-data[key][21:31]).mean())/B.dt)
        tuning.append(row)
    alpha_load = min(tuning, key=lambda r: r["load_mae_kw"])["alpha"]
    alpha_pv = min(tuning, key=lambda r: r["pv_mae_kw"])["alpha"]
    ridge, fit_log = forecasts(data, "ridge", (alpha_load, alpha_pv))
    dump_json(args.out/"forecast_fit_log.json", fit_log)
    pd.DataFrame(tuning).to_csv(args.out/"ridge_validation.csv", index=False)
    _, _, validation_soc = backtest(data, seasonal, 0, 21, B.initial, 0, "warmup", retain=False)
    warmup, _, evaluation_soc = backtest(data, seasonal, 21, 31, validation_soc, 0, "warmup", retain=False)
    warmup.to_csv(args.out/"warmup_last10days.csv", index=False)
    validation = []
    for method, pred in (("seasonal", seasonal), ("ridge", ridge)):
        for q in (0.0, 0.7, 0.8, 0.9):
            daily, _, _ = backtest(data, pred, 21, 31, validation_soc, q, method, retain=False)
            validation.append({"forecast": method, "quantile": q, "cost_yuan": float(daily.total_cost_yuan.sum()),
                              "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
                              "end_soc_kwh": float(daily.soc_end_kwh.iloc[-1])})
    selected = min(validation, key=lambda r: r["cost_yuan"])
    pd.DataFrame(validation).to_csv(args.out/"policy_validation.csv", index=False)
    stop = 38 if args.stage == "smoke" else 365
    strategies = [("seasonal_q0", "seasonal", 0.0), ("ridge_q0", "ridge", 0.0)]

```

```

for method in ("seasonal", "ridge"):
    best = min((v for v in validation if v["forecast"]==method), key=lambda r: r["cost_yuan"])
    if best["quantile"]:
        strategies.append((f"{method}_q{best['quantile']:g}", method, best["quantile"]))
selected_name = f"{selected['forecast']}_q{selected['quantile']:g}"
all_daily, metrics = [], []
for name, method, q in strategies:
    pred = {"seasonal": seasonal, "ridge": ridge}[method]
    daily, trace, _ = backtest(data, pred, 31, stop, evaluation_soc, q, name)
    all_daily.append(daily)
    trace.to_csv(args.out/f"{name}_intervals.csv.gz", index=False, float_format="%.17g")
    audit = audit_trace(trace, initial=evaluation_soc)
    row = {"strategy": name, "forecast": method, "quantile": q, "selected_on_january": name==selected_name,
           "days": len(daily), "total_cost_yuan": float(daily.total_cost_yuan.sum()),
           "normal_cost_yuan": float(daily.normal_cost_yuan.sum()),
           "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
           "emergency_kwh": float(daily.emergency_kwh.sum()), "emergency_intervals": int(daily.emergency_intervals.sum()),
           "plan_kwh": float(daily.plan_kwh.sum()), "spill_kwh": float(daily.spill_kwh.sum()),
           "solve_seconds": float(daily.solve_seconds.sum()), "initial_soc_kwh": evaluation_soc,
           "final_soc_kwh": float(daily.soc_end_kwh.iloc[-1]), "audit": audit}
    for key in ("load", "pv"):
        error = (pred[key][31:stop]-data[key][31:stop])/B.dt
        row[key+"_mae_kw"], row[key+"_rmse_kw"] = float(np.abs(error).mean()), float(np.sqrt(np.mean(error**2)))
    metrics.append(row)
    daily = pd.concat(all_daily, ignore_index=True)
    daily.to_csv(args.out/f"q2_daily_metrics.csv", index=False)

dump_json(args.out/"q2_summary.json", {"selection": selected, "selected_strategy": selected_name, "strategies": metrics})
command = f'python -m src.q12 --data-root "{args.data_root}" --out "{args.out}" --stage {args.stage}'
manifest = {"stage": args.stage, "battery": asdict(B), "time_interpretation": "interval mean power, right-end labels",
            "alpha_load": alpha_load, "alpha_pv": alpha_pv, "validation_start": "2025-01-22", "validation_end": "2025-01-31",
            "evaluation_start": "2025-02-01", "evaluation_end": str(data["dates"][stop-1].date()),
            "validation_initial_soc": validation_soc, "evaluation_initial_soc": evaluation_soc,
            "terminal_nominal_soc_minimum": 6000, "refit_period_days": 7, "training_window_days": 60,
            "residual_window_days": 28, "pool_halfwidth_slots": 3, "seed": 2026, "deterministic": True,
            "selected_strategy": selected_name, "command": command, "total_seconds": time.perf_counter()-start,
            "python": platform.python_version(), "numpy": np.__version__, "scipy": scipy.__version__, "pandas": pd.__version__,
            "platform": platform.platform(), "git_head_before_run": subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).strip(),
            "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(), "input_files": input_audit["input_files"]}
dump_json(args.out/"run_manifest.json", manifest)
print("COMPLETE", json.dumps({"selected": selected_name, "seconds": manifest["total_seconds"], "days": stop-31}), flush=True)

if __name__ == "__main__":
    main()

```

src/q34_data.py

```
"""Strict attachment 3/4 parsing and causal hourly forecast alignment."""
from pathlib import Path
import hashlib
import numpy as np
import pandas as pd
import openpyxl
from src.q12 import read_inputs, time_minutes, dump_json

def read_extended(root, audit_path=None):
    root=Path(root); data, base_audit=read_inputs(root)
    wb=openpyxl.load_workbook(root/'附件/附件 3.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==1461 and len(rows[0])==26
    assert list(rows[0][2:]) == [f'预报{i}小时' for i in range(1,25)]
    hourly=np.empty((365,4,24)); filled=0
    for day in range(365):
        date=data['dates'][day]
        for version in range(4):
            row=rows[1+day*4+version]
            if version==0:
                assert pd.Timestamp(row[0])==date
            elif row[0] in (None,''):
                filled+=1
            else: assert pd.Timestamp(row[0])==date
            assert time_minutes(row[1])==version*360
            hourly[day,version]=np.asarray(row[2:],float)
    wb=openpyxl.load_workbook(root/'附件/附件 4.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values); wb.close()
    assert len(rows)==366 and len(rows[0])==145
    assert [time_minutes(v) for v in rows[0][1:]]==list(range(10,1441,10))
    assert pd.DatetimeIndex([r[0] for r in rows[1:]]).equals(data['dates'])
    prices=np.asarray([r[1:] for r in rows[1:]],float)
    checks={}
    for name, values in [(('hourly_pv_kw', hourly), ('actual_price', prices))]:
        assert np.isfinite(values).all(), f'{name}: missing/nonfinite'
        assert (values==0).all(), f'{name}: negative values need investigation'
        if name=='actual_price': assert (values>0).all()
        q1,q3=np.quantile(values, [.25, .75]); iqr=q3-q1
        checks[name]={ 'shape':list(values.shape), 'count':values.size, 'missing_nonfinite':0,
            'negative':0, 'zero':int((values==0).sum()), 'min':float(values.min()), 'max':float(values.max()),
            'iqr_flags_retained':int(((values<q1-1.5*iqr)|(values>q3+1.5*iqr)).sum()),
            'deleted':0, 'imputed_numeric':0, 'winsorized':0, 'smoothed_observations':0}
    # An hourly point is an instantaneous forecast. Integrate its piecewise-linear
    # curve exactly over 10-minute cells. At lead 0 use the newest completed PV
    # interval (causally known) as a boundary proxy; midnight Jan 1 uses zero.
    pv=np.full((365,4,144), np.nan)
    for day in range(365):
        for version in range(4):
            start=version*36
            anchor=(data['pv'][day,start-1]*6 if start else
                data['pv'][day-1,-1]*6 if day else 0.0)
            vals=np.r_[anchor, hourly[day,version]]
            edges=np.arange(145-start)/6
            powers=np.interp(edges, np.arange(25), vals)
            pv[day,version,start:]=(powers[:-1]+powers[1:])/12
    data.update(hourly_pv=hourly, pv_issued=pv, actual_price=prices)
    audit={'pass':True, 'checks':checks, 'date_forward_fills_structural_only':filled,
        'dates':365, 'unique_issues':1460, 'issue_hours':[0,6,12,18],
        'forecast_horizons_hours':list(range(1,25)),
        'alignment':'Hourly lead endpoints; piecewise-linear integration over 10-minute cells; lead-zero
anchor from last completed observed PV interval; only current-day suffix traded.',
        'input_files':base_audit['input_files']+[
            {'path':name, 'sha256':hashlib.sha256((root/name).read_bytes()).hexdigest()}
            for name in ['附件/附件 3.xlsx', '附件/附件 4.xlsx']],
        'unusable_past_slots':'NaN intentionally marks targets before each issue; never supplied to a
solver',
        'auditor_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
    if audit_path: dump_json(audit_path, audit)
    return data, audit

if __name__=='__main__':
```

```
import argparse, json
p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
p.add_argument('--out',default='artifacts/q34/input_audit.json');args=p.parse_args()
Path(args.out).parent.mkdir(parents=True,exist_ok=True)
_,audit=read_extended(args.data_root,args.out)
print(json.dumps(audit,ensure_ascii=False,indent=2))
```

src/q4_forecast.py

"""Direct 24-hour price forecasts at four issue times, with monthly causal fits.

Seasonal, ridge and GRU see the same day/week lag curves and calendar. GRU predicts a residual around the seasonal curve, not future observed prices.

```
"""
from pathlib import Path
import argparse
import copy
import hashlib
import json
import time
import numpy as np
import pandas as pd
import torch
from torch import nn
from src.q12 import ridge_fit, dump_json
from src.q34_data import read_extended

torch.set_num_threads(1)

def calendar(indices):
    hours=(indices%144+.5)/144
    dow=(indices//144+2)%7 # 2025-01-01 Wednesday
    return np.stack([np.sin(2*np.pi*hours),np.cos(2*np.pi*hours),
                    np.sin(2*np.pi*dow/7),np.cos(2*np.pi*dow/7)],axis=-1)

def features_at(flat,origin):
    assert origin>=1008
    target=np.arange(origin,origin+144)
    return np.column_stack([flat[target-144],flat[target-1008],calendar(target)]).astype('float32')

def ridge_features(x):
    a,b=x[...,:0],x[...,:1]
    context=np.stack([a.mean(-1),b.mean(-1),a.std(-1),b.std(-1)],axis=-1)
    context=np.broadcast_to(context[...,:None,:],(*a.shape,4))
    return np.concatenate([x,np.stack([a-b,a*b],axis=-1),context],axis=-1)

class PriceGRU(nn.Module):
    def __init__(self):
        super().__init__();self.gru=nn.GRU(6,32,batch_first=True);self.head=nn.Linear(32,1)
        nn.init.zeros_(self.head.weight);nn.init.zeros_(self.head.bias)
    def forward(self,x):
        h,_=self.gru(x)
        return .5*(x[:, :, :0]+x[:, :, :1])+self.head(h).squeeze(-1)

def train_gru(x,y,origins,cutoff,seed,max_epochs=50):
    train=(origins+144<=cutoff-3*144);valid=(origins>=cutoff-3*144)
    assert train.sum()>0 and valid.sum()>0
    torch.manual_seed(seed);torch.use_deterministic_algorithms(True)
    mean=float(x[train,:, :2].mean());scale=max(float(x[train,:, :2].std()),1e-6)
    z=x.copy();z[:, :, :2]=(z[:, :, :2]-mean)/scale;target=(y-mean)/scale

    tx=torch.tensor(z[train]);ty=torch.tensor(target[train]);vx=torch.tensor(z[valid]);vy=torch.tensor(target[valid])
    model=PriceGRU();opt=torch.optim.Adam(model.parameters(),lr=.003)
    best=float('inf');weights=None;stale=0;history=[];rng=np.random.default_rng(seed)
    for epoch in range(max_epochs):
        model.train();order=rng.permutation(len(tx));losses=[]
        for left in range(0,len(order),32):
            idx=order[left:left+32];opt.zero_grad()
            loss=nn.functional.smooth_l1_loss(model(tx[idx]),ty[idx])
        loss.backward();nn.utils.clip_grad_norm_(model.parameters(),1.0);opt.step();losses.append(float(loss.detach()))
    model.eval()
```

```

        with torch.no_grad():score=float((model(vx)-vy).abs().mean())*scale
history.append({'epoch':epoch+1,'training_huber':float(np.mean(losses)), 'inner_validation_mae':score})
    if score<best-1e-7:best=score;weights=copy.deepcopy(model.state_dict());stale=0
    else:stale+=1
    if stale>=6:break
model.load_state_dict(weights);model.eval()
return model,mean,scale,history,int(train.sum()),int(valid.sum())

def same_day_errors(pred,actual,start,stop):
    values=[]
    for day in range(start,stop):
        for v in range(4):values.extend(pred[day,v,:144-v*36]-actual[day,v*36:])
    a=np.asarray(values)
    return {'mae':float(np.abs(a).mean()), 'rmse':float(np.sqrt(np.mean(a*a))), 'targets':len(a)}

def main():
    p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
    p.add_argument('--out',type=Path,default=Path('artifacts/q34'));p.add_argument('--
smoke',action='store_true')

args=p.parse_args();out=args.out;out.mkdir(parents=True,exist_ok=True);(out/'models').mkdir(exist_ok=True)
    data,_=read_extended(args.data_root);flat=data['actual_price'].ravel();started=time.perf_counter()
    stop=38 if args.smoke else 365
    origins=np.arange(1008,len(flat)-143,36)
    x=np.stack([features_at(flat,t) for t in origins]);y=np.stack([flat[t:t+144] for t in
origins]).astype('float32')
    # Fit on Jan 15/22 then each month boundary; never use targets ending after fit time.
    fit_days=[d for d in range(14,stop) if d in [14,21] or data['dates'][d].day==1]
    result={name:np.full((stop,4,144),np.nan,dtype='float64') for name in
['seasonal','ridge1','ridge10','ridge100','gru17','gru42','gru2026']}
    all_logs=[]
    for day in range(7,stop):
        for v in range(4):
            sample=features_at(flat,day*144+v*36)
            result['seasonal'][day,v]=sample[:,2].mean(-1)
    for fi,day in enumerate(fit_days):
        cutoff=day*144;end=fit_days[fi+1] if fi+1<len(fit_days) else stop
        choose=(origins+144<=cutoff)&(origins>=max(1008,cutoff-60*144))
        xx=x[choose];yy=y[choose];oo=origins[choose]
        assert (oo+144<=cutoff).all()
        future_x=np.stack([features_at(flat,d*144+v*36) for d in range(day,end) for v in range(4)])
        for alpha in [1.,10.,100.]:
            a=ridge_features(xx).reshape(-1,12);b=yy.ravel()
            model=ridge_fit(a,b,alpha);mean,scale,coef,intercept=model
            forecasts=((ridge_features(future_x)-mean)/scale)@coef+intercept
            result[f'ridge{alpha:g}'][day:end]=np.maximum(forecasts.reshape(end-day,4,144),.001)
        for seed in [17,42,2026]:
t=time.perf_counter();model,mean,scale,history,nt,nv=train_gru(xx,yy,oo,cutoff,seed+day,max_epochs=12 if
args.smoke else 50)
            z=future_x.copy();z[:,2,:]=z[:,2,:]-mean/scale
            with torch.no_grad():values=model(torch.tensor(z)).numpy()*scale+mean
            result[f'gru{seed}'][day:end]=np.maximum(values.reshape(end-day,4,144),.001)
            checkpoint=out/'models'/f'gru{seed}_fit{day:03d}.pt'
            torch.save({'state_dict':model.state_dict(),'mean':mean,'scale':scale,'fit_day':day,'seed':seed,
                'training_last_target_exclusive':cutoff,'architecture':'GRU(6,32)+Linear(32,1)+seasonal
residual'},checkpoint)

log={'fit_day':day,'fit_date':str(data['dates'][day].date()),'seed':seed,'actual_rng_seed':seed+day,

'training_windows':nt,'inner_validation_windows':nv,'last_target_exclusive':int((oo+144).max()),
    'seconds':time.perf_counter()-t,'history':history,'checkpoint':str(checkpoint)}
    all_logs.append(log)
    print(f'price GRU seed={seed} fit={day}: {len(history)} epochs, {log["seconds"]:.1f}s, best
inner MAE={min(r["inner_validation_mae"] for r in history):.4f}',flush=True)
    np.savez_compressed(out/'price_forecasts_checkpoint.npz',**result)
    dump_json(out/'price_fit_log.json',all_logs)
    for name in result:
        if name!='seasonal':result[name][7:14]=result['seasonal'][7:14]
        result['gru_mean']=np.mean([result[f'gru{s}'] for s in [17,42,2026]],axis=0)
        validation={name:same_day_errors(values,data['actual_price'],21,31) for name,values in result.items()}
        ridge_name=min(['ridge1','ridge10','ridge100'],key=lambda n:validation[n]['mae'])
        result['ridge']=result[ridge_name]
        chosen=min(['seasonal','ridge','gru_mean'],key=lambda n:validation[ridge_name if n=='ridge' else

```

```

n]['mae'])
    metrics={name:same_day_errors(result[name],data['actual_price'],31,stop) for name in
['seasonal','ridge','gru17','gru42','gru2026','gru_mean']}
    for name in ['seasonal','ridge','gru17','gru42','gru2026','gru_mean']:assert
np.isfinite(result[name][21:]).all()
    np.savez_compressed(out/'price_forecasts.npz',**result)

dump_json(out/'price_forecast_summary.json',{'validation':validation,'evaluation':metrics,'ridge_selected':r
idge_name,
        'selected_by_validation_price_mae':chosen,'seeds':[17,42,2026],'torch_version':torch.__version__,
        'epochs_max':12 if args.smoke else 50,'fit_days':fit_days,'elapsed_seconds':time.perf_counter()-
started,
        'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
        'evaluation_note':'Same-day suffix only, at each issue time; repeated targets at different origins
counted separately.',
        'training_rule':'Whole 24h targets end before fit; last 3 days internal validation; refit monthly
from scratch, last 60 days; Jan15/22 initialization.',
        'clip_floor':.001,'architecture':'Direct aligned day/week lag sequences plus known calendar;
residual GRU(6,32), no future teacher forcing'})

if __name__=='__main__':main()

```